

BOUNDFin: Static Multi-Resource Cost Semantics and an AOT-Wasm Translation Specification for Stateless Financial Kernels

Anonymous Author(s)

Abstract

Unbounded control flow, dynamic allocation, and unbounded inputs prevent finite pre-execution resource bounds for general contract programs. This paper defines BOUNDFin, a pure first-order language for stateless financial-contract kernels with explicit capacities, a capacity-indexed numeric fold, an immutable bounded array builder, and an acyclic call graph. A size abstraction distinguishes exact runtime lengths, symbolic upper lengths, and declared capacities. The scalar source cost semantics separately accounts for abstract operations, logical allocation events, cumulative reserved bytes, peak live scratch bytes, semantic reads and writes, decisions, and call depth. Termination and finite capacity instantiation are established for the defined core. Static-bound soundness remains an indexed proof obligation because the mutual footprint, error-prefix, and call-summary arguments are not closed. The translation design introduces a typed explicit-control-flow IR, reachability-based source liveness, checked frame-slot reuse, a 32-bit Wasm layout, and explicit source-IR and IR-Wasm proof obligations. The empirical plan distinguishes matched-scalar and production-optimised comparisons on three bounded financial kernels from a K1/K2-only explicit-SIMD ablation, and it decomposes the host-to-Wasm call boundary. No compiler, mechanised preservation proof, measurement, or performance result is reported.

1 Introduction

Small pricing, aggregation, and order-admission routines can form the computational boundary of a stateless financial contract: validated inputs enter, a value/decision or declared error leaves, and no persistent contract state changes. Repeated invocation under a latency budget makes both functional correctness and resource behaviour relevant. This paper uses “contract” for that pure invocation boundary; the selected routines are ordinary numerical/decision kernels when detached from it.

Testing a general-purpose program cannot establish a finite resource upper bound for all possible inputs. General recursion and unbounded loops may not terminate; unbounded aggregates prevent finite capacity substitution; dynamic allocation makes both cumulative demand and peak liveness runtime properties; and external state/calls add uncontrolled observations. Rich resource-aware systems can reason about some of these features [4, 14, 20]. BoundFin instead studies a deliberately smaller point: every iteration count and aggregate has a literal capacity, calls are acyclic, construction is immutable, and all effects are local.

The language must nevertheless express the experiment. A count-bounded fold supports scalar accumulation and nested finite control. An immutable builder produces a runtime-length array within a compile-time capacity. Together they express binomial backward induction without a mutable source array. A separate size abstraction propagates exact lengths when known, conservative symbolic upper lengths otherwise, and capacity-only bounds as a final fallback.

Memory requires more than summing allocations. A sequence can allocate many short-lived rows while using only two simultaneously. The source semantics therefore defines reachability and object death, while the analysis distinguishes allocation events, cumulative reserved bytes, and peak live scratch. Translation may reuse a frame slot only after a liveness certificate removes the old object from the source–target injection. This yields a specification-level connection between source footprint and BIR/Wasm frame layout without introducing general mutable arrays.

WebAssembly is a useful target because its typed structured control and linear memory have precise formal semantics [12] and mechanised core accounts [34, 35]. Those results do not verify a new compiler. BoundFin therefore defines proposed source–BIR and BIR–Wasm relations and records simulation, fault-exclusion, and observable preservation as open proof obligations. Correctness of the selected AOT Wasm-to-native backend remains a separate trust and experimental issue [31].

The same separation applies to cost. A scalar source theorem target would bound formal source events and reachability-defined scratch; a future lowering envelope could bound an abstract target trace. Neither entails cycles or latency. Generated instructions, caches, branch prediction, runtime trampolines, ABI work, frequency, and operating-system interference remain empirical. The methodology consequently distinguishes formal status, empirical correctness gates, and performance estimates.

1.1 Question and positioning

The central question is whether explicit capacities, size/footprint transformers, and scalar cost semantics can support a sound static-bound analysis, whether a future AOT source-to-Wasm implementation can discharge the stated observation-preservation obligations, and how that implementation compares with native C++ under matched scalar and production-optimised configurations. SIMD is a target-only ablation for K1 and K2; K3 remains scalar in the explicit-ablation matrix.

Prior work already covers restricted resource-bounded contracts, parametric gas synthesis, mechanised cost models, verified compilation, Wasm semantics, and Wasm performance [4, 7, 19, 21, 24]. The contribution claimed here is therefore not “the first paper containing each ingredient.” It is a unified specification in which bounded builders/folds, exact/upper/capacity sizes, peak-live immutable storage, changing memory injections, scalar Wasm lowering, and a financial-kernel evaluation boundary are defined together and made mutually checkable.

1.2 Specification-level contributions

The manuscript contributes:

1. a self-contained scalar BoundFin calculus with a numeric bounded fold, immutable bounded builder, functional primitive/error table, count certificates, size shapes, and 32-bit capacity/address admissibility;
2. rooted operational semantics and a candidate multi-resource analysis separating cumulative reservation from peak live scratch, together with an indexed static-bound proof obligation;
3. a mathematical BIR/Wasm translation design with capability-restricted writes, checked frame reuse, caller-owned aggregate returns, changing source–target injections, and explicit simulation and fault-exclusion obligations; and
4. a preregisterable methodology and implementation hand-off that keep correctness gates separate from proof, compare matched scalar and production-optimised artefacts separately, treat kernel

identity as confirmatory, restrict explicit SIMD to K1/K2, control K3 branch patterns, strengthen tail replication, and decompose the Wasm call boundary.

No compiler, mechanisation, or benchmark is reported. Every open formal claim is labelled as a proof obligation, and every performance conclusion remains result dependent.

1.3 Organisation

Section 2 positions the specification. Section 3 fixes the execution and trust boundaries. Sections 4 to 6 define source language, liveness, and analysis. Sections 7 and 8 define translation and proof obligations. Section 9 specifies future evaluation. The formal rule set appears in sections A and B.

2 Background and Related Work

The relevant literature establishes different kinds of evidence: language restrictions may imply finitary evaluation; resource systems may prove a bound relative to a cost semantics; compilation work may establish a simulation; and performance studies measure a particular runtime boundary. These results are complementary but not interchangeable.

2.1 Restricted contract languages

Simplicity removes loops and recursion from typed combinators and supplies denotational and Bit Machine models intended to support pre-execution time/space reasoning [25]. SCUR is more directly concerned with deployment-time resource upper bounds: it excludes unbounded loops/recursion, composes operation and branch costs, distinguishes reusable stack resources through frame/depth accounting, and reports a preliminary prototype [19]. SCUR therefore prevents any claim that bounded contract resource accounting is new in itself. Its blockchain runtime, affine function-pointer mechanism, and abstract physical-resource formulas differ from the pure array language, explicit peak-live object graph, and source-to-Wasm relation specified here.

Scilla separates pure computation from state transition and communication [29]; Nomos combines linear/session types and automatic amortised analysis for communicating contracts [11]; and a formal model of Algorand captures a stateless authorization-expression boundary [3]. These works motivate explicit effect boundaries, but their transaction, state, or protocol semantics differ from bounded numerical and decision kernels. The wider contract-language design space likewise shows that resource accounting depends on choices about communication, atomicity, state, and assets [28].

2.2 Static resource analysis and cost models

Gastap derives parametric EVM gas bounds [2], GASOL applies multiple cost models to analysis and optimisation [1], and exact gas-cost analysis proves a Hoare-style resource system against an explicit gas semantics [10]. Asparagus synthesises linear and polynomial parametric gas upper bounds using algebraic-geometric certificates and establishes completeness results for its template setting; its implementation was evaluated on deployed Solidity contracts [4]. These works are substantially more general in bound synthesis than BoundFin’s finite bounded sums. They target gas/contract analysis rather than a peak-live immutable-object semantics and source-to-Wasm simulation.

Automatic amortised analysis derives polynomial bounds by potential [13, 14]; certified variants produce compositional derivations [6]. Indexed and liquid-resource types expose value/size invariants [20, 33]; recurrence extraction and relational systems provide denotational or comparative cost views [8, 18]. A recent HOL4 formalisation establishes a reasonable time-and-space cost model for

call-by-push-value via simulations with abstract machines and Turing machines [7]. Its mechanised complexity result reinforces the need to state what a cost model counts, but it does not supply the object-capacity/peak-memory analysis or Wasm translation relation needed here.

High-level events require a separate link to machines. Learned cost semantics calibrates coefficients experimentally for a fixed compiler/platform [9]. End-to-end stack-bound verification instead preserves a particular quantitative trace property through compilation [5]. These precedents motivate the separation between scalar source soundness, optional target-event envelopes, and measured time.

2.3 Wasm semantics and compilation assurance

WebAssembly provides typed structured control, modules, and linear memory [12]. Isabelle and Coq mechanisations prove properties of core Wasm versions and supply executable components [34, 35]; they do not verify a new source compiler. CompCert illustrates pass-wise machine-checked simulation [21], while DeepSEA applies verified compilation to contracts [30]. Alive2 provides bounded translation validation rather than a universal compiler proof [22]. Tezla and WAMI show the analytical/engineering value of explicit intermediate representations, but the inspected versions do not discharge this paper’s source-to-Wasm obligations [17, 27]. Crocus further shows that Wasm-to-native instruction selection is a separate, operation- and backend-specific trust boundary [31].

2.4 Wasm performance and financial kernels

Native-relative Wasm performance is workload, runtime, compilation-mode, and architecture dependent. Browser-era work links gaps to generated code [15]; cross-architecture evaluation separates execution/startup and target effects [16]. Moron and Wallentowitz compare embedded Wasm runtimes with native and other virtual machines, including memory characteristics and the effect of AOT/runtime options [24]; this supersedes the earlier position-level evidence in the supplied corpus [32]. Lumos decomposes serverless lifecycle, I/O/serialisation, compute, cold/warm, and runtime effects across interpreted and AOT Wasm [23]. Its serverless/container boundary is not the present low-latency contract call, but its decomposition principle directly motivates the D0–D5 diagnostic design.

The inspected Black–Scholes study shows that vectorisation, layout, precision, NUMA, and transfer policy can materially alter financial-kernel comparisons [26]. It supplies no Wasm result for this paper. Accordingly, matched scalar, production-optimised, and K1/K2 explicit-SIMD comparisons are distinct; K3 has no explicit-SIMD treatment.

2.5 Positioning and evidence status

Table 1 records what representative inspected works establish, without mixing those completed publications with the present plan. The contribution here is best described as integrative: one specification connects capacity-indexed builders/folds, exact/upper/capacity size transformers, cumulative and peak-live resource summaries, changing memory injections with certified reuse, a scalar BIR-to-Wasm proof plan, and a financial-kernel experiment boundary. Those technical choices are concrete, but the manuscript does not claim that each individual idea is unprecedented or that the target results are proved.

Table 1: Status of representative prior contributions in the inspected sources. “Proved” and “mechanised” are used only when the cited work reports that evidence; absence refers only to the compared dimension.

Work	Restricted/resource contribution	Compilation or evaluation contribution	Evidence status in cited work
Simplicity [25]	Finitary language and static resource model	No Wasm/native financial comparison	Specified with formal arguments; implemented ecosystem outside compared claim
SCUR [19]	Static multi-resource formulas, including reusable stack treatment	Preliminary contract/runtime prototype; no Wasm preservation	Specified and prototyped; no mechanised source-to-Wasm proof claimed
Nomos [11]	Resource-aware session types and gas bounds	Contract-language implementation/evaluation, not Wasm translation	Specified and proved at its type-system level; implemented/evaluated
Asparagus [4]	Parametric gas-bound synthesis with soundness/completeness results for templates	Solidity tool and large contract evaluation	Proved, implemented, and evaluated; not a bounded source language/backend proof
CBPV cost model [7]	Reasonable time/space cost model	Abstract-machine/Turing-machine simulations	Proved and mechanised in HOL4; no Wasm/financial evaluation
Wasm mechanisations [34, 35]	No source DSL bound	Core Wasm semantic/type-soundness foundation	Proved and mechanised; not this lowering
CompCert and DeepSEA [21, 30]	Resource bounds not the compared main result	Semantics-preserving compilation	Proved/mechanised for their stated languages and targets
WAMI [17]	No source resource theorem	Wasm-oriented IR pipeline and performance study	Implemented and evaluated as a preprint; no complete preservation proof claimed
Wasm studies [15, 23, 24]	No source bound theorem	Runtime/native/lifecycle performance	Implemented and evaluated for their configurations; not formal compiler proofs

Table 2: Current BoundFin evidence ledger. “Specified” is not used as a synonym for proved, implemented, or evaluated.

Component	Specification	Formal evidence	Artifact evidence
Language and count fragment	mathematical core defined	termination proved; type/size obligations open	not implemented
Scalar candidate cost and peak bound	judgment and recurrences defined	resource-soundness obligation open	not evaluated
Source–BIR–Wasm relation	mathematical translation design	simulation/fault obligations open	not implemented
Matched scalar comparison	protocol specified	not a formal claim	not evaluated
Production-optimised comparison	protocol specified	not a formal claim	not evaluated
K1/K2 explicit-SIMD ablation	target protocol specified	separate proof obligation open	not implemented/evaluated

3 Problem Statement and Design Requirements

3.1 Contract execution model

We model a contract invocation as a closed, first-order function

$$f : \tau_1 \times \cdots \times \tau_k \rightarrow \tau_r$$

evaluated once for an invocation input. A scalar input has a fixed representation. An aggregate input has a static capacity N and a runtime length $\ell \leq N$. The host validates ABI shape, bounds, separation, and 32-bit endpoints before source evaluation. The function can read arguments and create bounded immutable local aggregates, but it cannot read a clock, global variable, file, network endpoint, random source, or persistent ledger state. It cannot call the host. Its observable result is either a value of τ_r or a declared error code.

Definition 3.1 (Stateless financial contract). *A stateless financial contract is a total source evaluation on ABI-valid bounded inputs whose outcome is unique up to fresh-object renaming. Floating primitives use the functional NaN convention below, while external observations identify all NaN encodings. “Stateless” means that two invocations share no mutable contract state. Local scratch storage exists only for the duration of an invocation and is not observable after return.*

This use of “contract” refers to the validated input/output computation boundary, not to distributed consensus or persistent smart-contract state. The pricing and aggregation routines are numerical kernels; they become contract kernels only when embedded in this declared invocation boundary. Stateless blockchain authorisation models provide a related separation [3], but the selected functions compute prices, aggregates, or decisions rather than transaction validity.

3.2 Observations, errors, and numeric behaviour

The scalar types are `bool`, `i32`, `i64`, and `f64`. Integer addition, subtraction, and multiplication are two’s-complement operations modulo 2^{32} or 2^{64} . Division and remainder return a declared error on zero divisor and on the signed minimum divided by -1 . Boolean and integer comparisons are exact.

Let $q_{\text{NaN}} = 0x7ff8000000000000$ and define $\text{can}(z) = q_{\text{NaN}}$ when the binary64 word z encodes any NaN, and $\text{can}(z) = z$ otherwise. Binary64 arithmetic uses round-to-nearest, ties-to-even and the fixed scalar Wasm operation, then applies `can`; unary floating operations are canonicalised in the same way. Floating comparisons use the Wasm Boolean result. Thus the source primitive function is single-valued even for NaNs. Source literals and ABI inputs may carry any 64-bit NaN word, but a floating primitive that returns NaN returns q_{NaN} . External observation identifies all NaN encodings; non-NaN words, including the signs of zero, must agree bitwise. The scalar core contains no transcendental or fused operation.

Declared errors are `Bounds`, `DivZero`, `DivOverflow`, and `InvalidABI`. Source evaluation is left-to-right and stops at the first error. A validated well-typed program cannot produce `InvalidABI` internally; that outcome belongs to the invocation boundary. Bounds errors use status returns, not target traps. The complete primitive cases are normative in table 5; no unlisted numeric operation is assumed.

3.3 Memory and trust boundaries

Source arrays are immutable values. The semantics records object graphs in an invocation-local store and defines liveness by reachability from continuations, environments, partial literal/builder

constructions, accumulators, and results. Simultaneously live objects occupy distinct abstract capacity regions; unreachable objects are irrelevant modulo rooted-store equivalence. This permits a target frame slot to be reused only after a certificate proves the old object dead. No address escapes, and source programs cannot inspect addresses or partial construction storage.

Cumulative reserved bytes and peak live scratch bytes are distinct. The former counts every local capacity reservation, including an object later made unreachable; the latter is the maximum capacity-region sum simultaneously reachable during evaluation. Input/output ABI regions, stacked BIR frames, and total Wasm linear memory are reported separately.

The trust boundary for the preservation verification goal includes conformance of the finite-derivation checker, primitive table, ABI validator, Wasm validator/specification, and stated host assumptions. Certificate generation may use an untrusted solver, but module acceptance depends on checking a finite derivation in the rules of section A.4. The source-to-BIR and BIR-to-Wasm arguments remain proof obligations and the corresponding transformations would remain trusted until those obligations are discharged. The selected AOT runtime, native backend, operating system, and processor lie outside the source-to-Wasm relation. Verified compilation and Wasm instruction-selection work show why these components must be named [21, 31].

3.4 Meaning of predictable execution

Definition 3.2 (Language-level resource predictability). *For a well-typed program f and declared input-capacity environment Δ , execution is resource-predictable at the language level when (i) every admissible invocation terminates and (ii) a static expression can be evaluated under every admissible size environment to give a finite upper bound on every scalar source event component and peak-live scratch.*

This is the target property, not the present evidence status. Definitions 4.1 and 6.2 establish termination and finite evaluation of the candidate resource expressions, respectively; clause (ii) remains exactly definition 6.1. Until that obligation is discharged, analyser output is a finite candidate bound rather than an established upper bound. The definition also does not promise a fixed number of cycles, a deadline, or a latency distribution. Cache state, branch prediction, frequency management, runtime code generation, operating-system interference, and generated native instructions remain machine-level variables. Learned or target-specific calibration may relate abstract source events to observed time, but that relation is empirical [9].

3.5 Requirements and non-goals

The requirements below are specification criteria. They do not assert completed proofs, artefacts, or measurements.

4 DSL Syntax, Types, and Size Abstraction

BOUND_{FIN} is a first-order call-by-value expression language. Its core contains no implicit iteration or allocation: the only iterative constructs expose both a runtime count and a compile-time capacity. The normative lexical, abstract-syntax, typing, size, evaluation, and cost judgments are collected in section A; this section introduces their design.

Table 3: Design requirements and their verification criteria.

ID	Requirement	Criterion
R1	Total bounded evaluation	No recursion; acyclic calls; every fold/builder count has a static capacity proof.
R2	Explicit data sizes	Exact, upper, and capacity shapes cover every aggregate result and call.
R3	Pure invocation	No persistent state, external call, dynamic dispatch, nondeterministic input, or address observation.
R4	Defined numerics	Fixed-width integer and binary64 cases, errors, rounding, and NaN observation are explicit.
R5	Compositional cost	Rules distinguish operations, allocation events, cumulative reserved bytes, peak live scratch, semantic bytes, decisions, and depth.
R6	Checkable translation boundary	Typed BIR, write capabilities, liveness certificates, 32-bit layout, and a validated Wasm subset are specified.
R7	Functional preservation	Source and target return observably equivalent values or the same declared error under stated assumptions.
R8	Controlled evaluation	Matched-scalar and production-optimised comparisons cover K1–K3; the separately named explicit-SIMD matrix covers K1/K2 only, with equivalent contract boundaries.
N1	Not a cycle-accurate WCET model	No mapping from abstract operations to cycles is assumed.
N2	Not a blockchain protocol	Consensus, persistence, re-entrancy, networking, and asset custody are excluded.
N3	Not a general-purpose language	General recursion, dynamic allocation, closures, and unbounded containers are rejected.
N4	Not a completed verification or experiment	Open formal statements are proof obligations; all measurements are future work.

4.1 Types, modules, and expressions

Let $N \in \mathbb{N}$ be a literal capacity satisfying $0 \leq N \leq N_{\max} = 2^{31} - 1$. Scalar and aggregate types are

$$b ::= \text{bool} \mid \text{i32} \mid \text{i64} \mid \text{f64}, \quad \tau ::= b \mid \tau_1 \times \cdots \times \tau_k \mid \text{arr}\langle\tau, N\rangle.$$

An array value has a runtime length $\ell \leq N$ and immutable logical elements. Products have fixed arity. The core expression grammar is

$$\begin{aligned} e ::= & x \mid c \mid p(e_1, \dots, e_k) \mid \text{let } x = e_1 \text{ in } e_2 \mid \text{if } e_c \text{ then } e_t \text{ else } e_f \\ & \mid (e_1, \dots, e_k) \mid \pi_j(e) \mid [e_0, \dots, e_{m-1}]_N \mid \text{len}(e) \mid e_a[e_i] \mid f(e_1, \dots, e_k) \\ & \mid \text{fold}_N(e_n; i, x; e_0; e_b) \mid \text{build}_N(e_n; i.e_b). \end{aligned}$$

Primitive p ranges exactly over the scalar table in table 5. A literal $[e_0, \dots, e_{m-1}]_N$ contains m distinct expression occurrences and evaluates that finite vector once from left to right. It is not syntactic sugar for a builder. The numeric fold first evaluates count e_n and initial accumulator e_0 , then evaluates e_b exactly n times in increasing index order with $i \in [0, n)$ and x bound to the previous accumulator. The builder evaluates the single indexed body e_b once for each $i \in [0, n)$ and returns a fresh immutable $\text{arr}\langle\tau, N\rangle$ of length n . Both iterative constructs require a static derivation of $0 \leq n \leq N$; neither silently clamps an invalid count.

A module M is an ordered list of declarations

$$f(x_1:\tau_1, \dots, x_k:\tau_k) : \tau_r = e$$

and one exported entry point. We write $f \prec_M g$ exactly when declaration f occurs strictly before declaration g . The body of g may call only f with $f \prec_M g$; hence the call graph is acyclic. There is no **while**, unrestricted **for**, recursion, higher-order value, dynamic dispatch, exception handler, implicit heap allocation, mutation, or external call.

4.2 Typing and bounded counts

Size terms and upper-bound terms are

$$t ::= n \mid \xi \mid t + t \mid kt, \quad u ::= t \mid \max(u, u), \quad n, k \in \mathbb{N},$$

where ξ ranges over ABI lengths, count results, and bound indices. A constraint is $t = t$, $t \leq t$, or a finite conjunction of constraints; Δ is a finite set of them. A size environment $\eta : \text{fv}(\Delta) \rightarrow \mathbb{N}$ extends homomorphically to t and u , and $\eta \models \Delta$ has its usual mathematical-integer meaning. Judgment $\Delta \vdash_{\text{cert}} \phi$ means that a finite derivation of ϕ has been checked using assumption, constant normalisation, equality substitution, transitivity, addition, nonnegative scaling, and the introduction/elimination rules for max. Certificate checking is structural and terminates; certificate generation, including any SMT solver, is outside the trusted checker. A true formula for which no accepted certificate is supplied is conservatively rejected. The rule set and its soundness lemma are in section A.4.

The judgment $\Sigma; \Delta; \Gamma \vdash e : \tau$ uses function summaries Σ , the constraints Δ , and variable types Γ . The auxiliary judgment

$$\Sigma; \Delta; \Gamma \vdash_{\text{cnt}} e \Rightarrow (\lambda_n, u_n; \nu_n)$$

types e as i32, names its nonnegative signed interpretation ν_n , and establishes $0 \leq \nu_n \leq u_n \leq 2^{31} - 1$. Its exact component λ_n is either a term s_n , with a checked derivation of $\nu_n = s_n$, or $\star$. Constants, ABI length symbols, len, bound indices, conditionals, checked addition/subtraction, and calls have

the rules in section A.4. Addition or subtraction may enter the count fragment only when its certificate proves the corresponding mathematical result is in $[0, 2^{31} - 1]$; modular wrap is therefore never used as size arithmetic. A count is admissible for capacity N only when $\Delta \vdash_{\text{cert}} u_n \leq N$. Put $\Delta_n = \Delta \cup \{0 \leq \nu_n \leq u_n \leq N\}$ and additionally $\nu_n = s_n$ when $\lambda_n = s_n$. The two bounded rules are

$$\frac{\begin{array}{c} \Sigma; \Delta; \Gamma \vdash_{\text{cnt}} e_n \Rightarrow (\lambda_n, u_n; \nu_n) \quad \Delta \vdash_{\text{cert}} u_n \leq N \quad \Sigma; \Delta; \Gamma \vdash e_0 : \tau_x \\ \Sigma; \Delta_n, 0 \leq i < \nu_n; \Gamma, i:\text{idx}(N), x:\tau_x \vdash e_b : \tau_x \end{array}}{\Sigma; \Delta; \Gamma \vdash \mathbf{fold}_N(e_n; i, x; e_0; e_b) : \tau_x} \quad (\text{T-Fold})$$

$$\frac{\begin{array}{c} \Sigma; \Delta; \Gamma \vdash_{\text{cnt}} e_n \Rightarrow (\lambda_n, u_n; \nu_n) \quad \Delta \vdash_{\text{cert}} u_n \leq N \\ \Sigma; \Delta_n, 0 \leq i < \nu_n; \Gamma, i:\text{idx}(N) \vdash e_b : \tau \end{array}}{\Sigma; \Delta; \Gamma \vdash \mathbf{build}_N(e_n; i.e_b) : \text{arr}\langle \tau, N \rangle.} \quad (\text{T-Build})$$

General i32 expressions are not accepted as counts merely because their machine type is correct. Exactness improves parametric reporting but is not required when a conservative upper term is certified. Inside either body, $\text{idx}(N)$ carries $0 \leq i < \nu_n \leq u_n \leq N$ and is not an ABI type. An access justified by these facts may omit a dynamic check only after the same certificate is attached to the lowering; other indexing remains checked.

The call rule uses the declared order without reversing its meaning:

$$\frac{\Sigma(f) = (\vec{\tau} \rightarrow \tau, K_f, \Phi_f) \quad \Sigma; \Delta; \Gamma \vdash \vec{e} : \vec{\tau} \quad f \prec_M g}{\Sigma, g; \Delta; \Gamma \vdash f(\vec{e}) : \tau} \quad (\text{T-Call})$$

K_f is the result-size transformer and Φ_f is the cost/footprint transformer defined below and in section 6. All other typing rules are given in section A.5.

4.3 Exact, upper, and capacity size information

A capacity alone is insufficient for a parametric cost. Size shapes are therefore

$$\kappa ::= \text{scalar} \mid \text{prod}(\kappa_1, \dots, \kappa_k) \mid \text{array}(\lambda, u, N; \bar{\kappa}),$$

where λ is either an exact symbolic length s or $\star$ (not known exactly), u is a proved symbolic upper length, N is the declared capacity, and $\bar{\kappa}$ conservatively joins the shapes of every logical element. A valid shape satisfies $\Delta \models 0 \leq s \leq u \leq N$ when $\lambda = s$, and $\Delta \models 0 \leq u \leq N$ when $\lambda = \star$. Function $\text{capshape}(\tau)$ recursively replaces every array length in a type by its capacity and supplies a finite fallback for nested aggregate elements. The separate judgment

$$\Sigma; \Delta; \Gamma \vdash_{\text{sz}} e \Rightarrow \kappa$$

does not allocate or evaluate e ; it summarises result sizes.

ABI array argument $x:\text{arr}\langle \tau, N \rangle$ receives a fresh formal length n_x under $0 \leq n_x \leq N$. Its shape is

$$\text{array}(n_x, n_x, N; \text{capshape}(\tau)).$$

A literal of m elements joins its element shapes into $\bar{\kappa}$. A builder has

$$\text{array} \left(\lambda_n, u_n, N; \bigsqcup_{0 \leq i < u_n} \kappa_b(i) \right);$$

hence an upper-only count yields exact component $\star$ rather than being misreported as exact. A conditional preserves an exact length only when both branches establish the same expression;

otherwise it has exact component $\star$ and upper component $\max(u_t, u_f)$, and it joins element summaries recursively. Products retain component shapes, projection selects one, and indexing returns the array's element summary. A `let` substitutes the shape of its bound expression into the body judgment.

For a function with formal array lengths $\vec{n}$, $K_f(\vec{n})$ is checked with the body and stored in Σ . At a call, exact or upper actual length expressions are substituted simultaneously into K_f ; nested calls therefore do not invent fresh unrelated lengths. If an exact actual is unavailable, upper substitution yields a conservative $\star$ result. This distinction is retained in reports: $B(\vec{n})$ is a parametric bound, $B(\vec{u})$ is a conservative symbolic upper bound, and $B(\vec{N})$ is the capacity-only bound. The full rules, including products containing arrays, appear in table 8.

4.4 Well-formedness and rejection

Judgment $\vdash M \text{ wf}$ requires unique declarations, complete primitive signatures, a well-typed body and size transformer for every declaration, calls respecting $\prec_M$, finite representation shapes, and the module-memory admissibility predicate of section 7.5. A conforming front end rejects, with a stable category:

- unsupported syntax or primitive names;
- type, arity, projection, or branch-result mismatches;
- nonliteral, negative, or excessive capacities and any representation whose checked layout exceeds the 32-bit target limit;
- a fold/builder count without a refinement proving its runtime value between zero and N ;
- mutation, address observation, import, host access, general allocation, recursion, or a call not satisfying $f \prec_M g$;
- an unmodelled conversion, floating contraction, precision change, or transcendental; and
- an ABI length, header, alignment, separation, or region-end violation.

Rejected modules have no core-language evaluation. Invocation rejection yields `InvalidABI` before the entry body begins.

4.5 Termination and zero-allocation subset

Theorem 4.1 (Termination of the source calculus). *If $\vdash M \text{ wf}$ and the exported body is closed under an ABI-valid input, its evaluation has a finite derivation.*

Proof. Assign declaration D_j rank j . A call in D_j targets only D_i with $i < j$. We prove, by strong induction on rank, that evaluation of every expression in D_j terminates for every conforming finite environment and store. For a fixed rank, use structural induction on the expression. Variables, constants, and terminal actions are immediate. Ordered child evaluation invokes the hypothesis on finitely many proper subexpressions and stops either at the first error or after the finite list. A selected conditional branch and a let body are proper subexpressions. A call invokes a lower-rank declaration, so the outer induction applies.

For a literal, the finite vector has length $m \leq N$. Induction on the remaining suffix $m - j$ proves termination of its distinct-expression prefix relation: the base seals the prefix, while the step evaluates the syntactically fixed occurrence e_j by structural induction and either stops at its first

error or advances to $j + 1$. For a fold or builder, evaluation of the count terminates structurally and returns either an error or a word whose certified signed value is some n with $0 \leq n \leq N < \infty$. The initial expression, when present, terminates structurally. Induction on residual count $n - k$ proves termination of each iterative auxiliary relation: residual zero uses its base rule; a positive residual evaluates one body occurrence by the structural hypothesis and either stops on error or recurs with residual decreased by one. Store restriction and fresh-name choice are finite mathematical operations on a finite store and introduce no premise. These cases exhaust the grammar and the error-propagation relations of section A.6. $\square$

The source zero-allocation subset excludes array literals, builders, and any call whose summary may return a newly materialised aggregate. Input-array reads, scalar/product-of-scalars construction, conditionals, numeric folds with scalar/product-of-scalars accumulators, and qualifying calls remain permitted. “Zero allocation” means zero logical local-object events in the source semantics; it is not a claim about native prologues or runtime metadata.

5 Operational Semantics and Source Liveness

5.1 Values, stores, and reachability

Scalar values are fixed-width bit vectors; binary64 interpretation is fixed in section 3. Products are finite tuples. An array value is an object identifier α in an invocation-local store

$$\sigma(\alpha) = \langle \tau, N, \ell, \omega, v_0, \dots, v_{\ell-1} \rangle, \quad 0 \leq \ell \leq N,$$

where origin $\omega \in \{\text{abi}, \text{local}\}$ distinguishes caller-owned input from source-local scratch. Stored arrays are immutable and identifiers are not source values that can be compared or converted to integers. A private construction object may have an administrative initialisation prefix, but it becomes a source array only when sealed.

For a finite root collection V , $\text{Reach}_\sigma(V)$ is the least set containing every array identifier occurring in V and every identifier recursively occurring in the elements of a reached object. The reachable substore is $\sigma \upharpoonright V$. Two stores are equivalent at roots V , written $\sigma_1 \equiv_V \sigma_2$, when the rooted object graphs are isomorphic under a fresh-identifier bijection preserving types, capacities, lengths or prefixes, origins, scalar bits, product fields, and edges. Unreachable objects are deliberately absent from this equivalence.

Let

$$\text{live}_\sigma(V) = \sum_{\substack{\alpha \in \text{Reach}_\sigma(V) \\ \text{origin}(\alpha) = \text{local}}} \text{region}(\text{type}(\alpha), \text{cap}(\alpha))$$

be simultaneously live local scratch bytes. Each object is counted once even if aliased through several product fields. ABI input regions are excluded from source scratch but included separately in the target memory bound. The region function includes header, alignment, and capacity storage and is defined in section A.8.

5.2 Rooted big-step judgment

An environment ρ maps variables to values. Results are $r ::= \text{ok}(v) \mid \text{err}(\epsilon)$ for $\epsilon \in \{\text{Bounds}, \text{DivZero}, \text{DivOverflow}\}$; InvalidABI is produced only before entry. The evaluation judgment is

$$\Sigma; K \vdash \langle e, \rho, \sigma \rangle \Downarrow \langle r, \sigma' \rangle \mid C,$$

where K is the collection of continuation roots that must survive evaluation and C is the dynamic scalar source cost of section 6. Write

$$\text{rv}_\rho(X) = \{\rho(x) : x \in X \cap \text{dom}(\rho)\}.$$

Every premise explicitly retains K , the values of free variables needed by the unexecuted continuation, and already obtained operands. On successful return the store is restricted to $K \cup \{v\}$; on error it is restricted to K . This is semantic quotienting, not an observable collection event or a runtime-garbage-collector claim.

Evaluation is left-to-right. The parameter-complete sequence judgment in section A.6 retains, while child e_j executes, the original continuation roots, earlier operand values, and rv_ρ of the remaining suffix. Its four constructors specify the empty, successful-step, first-child-error, and later-error cases. No later child is evaluated after the first error. Primitive application uses the total function δ_p from typed operands to a value or one of the declared arithmetic errors.

For **let**, free-variable values needed by the body and the successful bound value remain roots until the body has returned. A conditional retains both potential branch environments while its guard executes and then evaluates only the selected branch. Array indexing evaluates array and index in order, reads the length header, and returns element j only when $0 \leq j < \ell$; otherwise it returns **Bounds**. A length operation returns the nonnegative length bits as **i32**. The normative rule schemas for every expression form are in section A.6.

5.3 Array literals

A literal $[e_0, \dots, e_{m-1}]_N$ reserves one fresh local capacity- N region before evaluating an element. Its distinct occurrences are evaluated once each in increasing position by

$$\Sigma; K \vdash \langle \vec{e}, \rho, \sigma, j, m, \alpha \rangle \Downarrow_{\text{lit}} \langle r, \sigma' \rangle \mid C.$$

At position j , the partial object α , its first j values, and free-variable values required by $e_j, \dots, e_{m-1}$ are retained. Success of e_j causes exactly one write to position j before the suffix begins. Its first error returns immediately and removes α and element graphs reachable only from that prefix. At $j = m$, one seal records logical length m and returns α . The literal relation has no source-level index conditional and contributes no loop-test decision. Its rules and exact trace are **LIT0**, **LITS**, and **LITERR** in section A.6.

5.4 Bounded numeric folds

After count e_n yields n and the static derivation guarantees $0 \leq n \leq N$, the fold evaluates e_0 and invokes

$$\Sigma; K_b \vdash \langle e_b, \rho, \sigma, i, x, k, n, v \rangle \Downarrow_{\text{fold}} \langle r, \sigma' \rangle \mid C, \quad K_b = K \cup \text{rv}_\rho(\text{FV}(e_b) \setminus \{i, x\}),$$

where k is the next index. Its rules are

$$\frac{k = n}{\Sigma; K_b \vdash \langle e_b, \rho, \sigma, i, x, k, n, v \rangle \Downarrow_{\text{fold}} \langle \text{ok}(v), \sigma \upharpoonright (K_b \cup \{v\}) \rangle \mid \chi_{\text{loopTest}}} \quad (\text{E-Fold0})$$

$$\frac{\begin{array}{l} k < n \quad \Sigma; K_b \cup \{v\} \vdash \langle e_b, \rho[i \mapsto k, x \mapsto v], \sigma \rangle \Downarrow \langle \text{ok}(v'), \sigma_1 \rangle \mid C_b \\ \Sigma; K_b \vdash \langle e_b, \rho, \sigma_1, i, x, k+1, n, v' \rangle \Downarrow_{\text{fold}} \langle r, \sigma_2 \rangle \mid C_r \end{array}}{\Sigma; K_b \vdash \langle e_b, \rho, \sigma, i, x, k, n, v \rangle \Downarrow_{\text{fold}} \langle r, \sigma_2 \rangle \mid \chi_{\text{loopTest}} \oplus C_b \oplus C_r} \quad (\text{E-FoldS})$$

$$\frac{k < n \quad \Sigma; K_b \cup \{v\} \vdash \langle e_b, \rho[i \mapsto k, x \mapsto v], \sigma \rangle \Downarrow \langle \text{err}(\epsilon), \sigma_1 \rangle \mid C_b}{\Sigma; K_b \vdash \langle e_b, \rho, \sigma, i, x, k, n, v \rangle \Downarrow_{\text{fold}} \langle \text{err}(\epsilon), \sigma_1 \upharpoonright K_b \rangle \mid \chi_{\text{loopTest}} \oplus C_b} \quad (\text{E-FoldErr})$$

The enclosing fold removes closure roots not in its original K after success or failure. The old accumulator is retained while the body computes the new one. It may leave the store at the iteration boundary only if unreachable from $K_b \cup \{v'\}$; a body may therefore retain the old accumulator inside its result, in which case peak analysis must include the retained chain.

5.5 Immutable bounded builders

After e_n yields n with $0 \leq n \leq N$, the builder reserves a fresh local region α with capacity N and prefix counter zero. It invokes the builder auxiliary with $K_b = K \cup \text{rv}_\rho(\text{FV}(e_b) \setminus \{i\})$. At step $j < n$, α , its first j values, and K_b are roots while the same indexed body e_b evaluates under $i \mapsto j$. A successful value is written once to position j and the prefix advances. At $j = n$, the header length is fixed to n , the object is sealed, and α is returned. Unlike a literal, this auxiliary repeats one body under successive index bindings and contributes a loop test at every step and at exit.

No source evaluation can observe a partially initialised builder. If count or body evaluation fails, strict propagation returns that error; the reserved region and element objects reachable only from it become dead. Their allocation and completed writes remain in cumulative dynamic cost but not in final live storage. A builder never overwrites an existing sealed array.

5.6 Calls and result stores

For $\Sigma(f) = (\vec{x}:\vec{\tau}, e_f)$, actuals are evaluated left-to-right. The body evaluates in a fresh environment $[\vec{x} \mapsto \vec{v}]$ with caller continuation roots and live actuals retained. An actual-expression error does not enter the callee and adds no call/return event. A body success or declared error adds $\chi_{\text{callReturn}}$ and lifts the callee's relative active depth by one. Aggregate source results remain object graphs; target lowering later realizes them in caller-owned result storage, so no pointer to a returned callee frame is required. The premise $f \prec_M g$ makes recursive evaluation well founded.

5.7 Metatheoretic status

Theorem 5.1 (Determinism modulo rooted-store isomorphism). *For fixed well-formed $\Sigma, K, e, \rho, \sigma$, any two derivations yield the same declared error or observably equivalent value, the same source event counts, the same peak-live byte count, and stores equivalent at the returned roots.*

Proof. Induct on the first derivation, with subsidiary inductions on ordered-sequence length, literal suffix length, and fold/builder residual count. The four sequence constructors fix either the first error position or all operand values because each premise carries the same Σ, K, ρ , suffix roots, and store restriction. Variable and constant results are fixed. For a primitive, the operand hypotheses and first error are fixed; δ_p is a function because floating results are canonicalised by `can`. Product, projection, length, and index terminal actions are consequently unique. Let `fixes` its bound result and declared body roots; a guard fixes exactly one branch.

For a literal, suffix induction fixes the distinct occurrence at position j and the first error, if any. For a builder, residual-count induction instead fixes repeated evaluations of its single body under $i = j$. In either construction, two fresh region choices extend the rooted-identifier bijection once; every successful position has one write and only the base case seals. Fold residual-count induction fixes the accumulator sequence. Sequence induction fixes actuals, and declaration order fixes the direct-call body and its result. Root restriction removes exactly identifiers outside the least reachable closure, so isomorphic declared root sets have isomorphic restrictions and the same region sum. Every rule therefore fixes cumulative events, active-depth lifting, and the maximum rooted peak. $\square$

Proof Obligation (PO-SIZE-SUBST: simultaneous shape and summary substitution). *Let θ simultaneously replace formal scalar size names, exact/upper array lengths, nested product/array shapes, retained-result footprints, and earlier-declaration summary parameters. Every exact formal maps either to a certified exact actual or to $\star$ together with a certified upper actual; capacities are unchanged. If the formal constraints and candidate shape/resource derivations are checked, then substitution must preserve their certified inequalities, recursively conservative element-shape joins, capacity fallback, and well-formed finite sums/recurrences. For an earlier declaration, the instantiated result shape and candidate footprint/resource transformer must be the capture-avoiding simultaneous instance of its stored summary.*

The structural proof closes scalar, product, nested-array, exact-to-exact, exact-to- $\star$, upper, capacity-fallback, maximum/join, and finite resource-expression cases. A certificate substitution node transports every formal inequality to the actual environment; when exactness is lost, no equality is transported and the upper inequality is retained. Induction on declaration rank makes syntactic instantiation of an earlier summary finite. The remaining semantic call case is mutual with PO-SIZE and PO-COST-CALL: it must show that the stored earlier summary already denotes every runtime result graph and rooted peak under the substituted actual aliases. This is missing proof exposition, not a known counterexample, so PO-SIZE-SUBST remains open.

Alias-overcount lemma. For every finite well-typed value graph described by shape κ , the bytes in the set of reachable local objects are no greater than $\text{capfoot}(\tau, \kappa)$, provided each immediate element graph satisfies its component shape. Counting the same identifier through two fields can make capfoot larger but cannot make it smaller.

Proof. Induct on the finite, nonrecursive type depth. Scalars reach no local object. For a product, its reachable-object set is a union of component sets, whose byte measure is at most the sum of their measures; duplicate identifiers reduce the union relative to that sum. For an array, the reachable set is its own local region, when local, union the sets reached through its logical elements. Its region is bounded by the capacity term and each element set by the induction hypothesis. Again, overlap between elements can only reduce the union relative to the defining sum. ABI-origin objects contribute zero to the measured local set while the candidate expression may still overcount them. $\square$

Declared-root closure lemma. Let S be a declared root set and $\sigma_S = \sigma \upharpoonright S$. For every value whose identifiers lie in $\text{Reach}_\sigma(S)$, lookup and every path reachable from that value are identical in σ and σ_S .

Proof. $\text{Reach}_\sigma(S)$ is closed under every identifier edge by its least-fixed-point definition. Restriction removes only identifiers outside that closed set and changes no retained object, so induction on path length gives identical lookup and reachability. $\square$

Proof Obligation (PO-LIVE: conditional root restriction). *At every restriction point, the declared set must contain K , values of all active free variables, earlier operands needed by the continuation, and the current accumulator, partial literal/builder, or pending result. Future evaluation may read an old identifier only through those retained values; allocations introduce fresh identifiers. Under that discipline, replacing σ by $\sigma \upharpoonright S$ must preserve every later outcome and event while permitting objects outside $\text{Reach}_\sigma(S)$ to die.*

The declared-root closure lemma handles each individual lookup. Sequence and literal rules now retain their complete suffix roots, and fold/builder entry retains body closures. The remaining proof is the mutual continuation argument for a returned aggregate that aliases an actual across a nested call or survives a builder failure through an outer root. No counterexample to the corrected rules is known. The unrestricted equation with arbitrary future root extension is false: if σ contains one object α , restriction to $S = \emptyset$ removes it, and subsequently adding root α cannot recover its contents.

Proof Obligation 5.2 (Type, size, and rooted-store preservation). *For a well-formed module and conforming rooted state, evaluation returns a declared error or a value of the declared type and size shape; every reached object respects its capacity, every sealed array is immutable, and the returned store contains every root required by the continuation or result graph.*

The proof attempt is a mutual induction on evaluation, typing, and size derivations. Primitive, product, projection, length, index, conditional, and binding cases follow from signatures and declared-root closure. Literal-prefix induction uses the distinct element typing premises, preserves a typed prefix after each one-time write, and seals exactly length m ; builder-prefix induction uses $n \leq N$ and one typed body result per index. Fold induction preserves the accumulator type and shape recurrence. The exact remaining cases are the semantic earlier-call instance of PO-SIZE-SUBST and the nested aggregate alias case of PO-LIVE. Consequently definition 5.2 remains an obligation; the corrected literal rules introduce no additional unresolved case.

The source remains stateless: the store represents finite invocation-local values, exposes no address or mutation operation, and is discarded after result serialisation. Reachability states peak memory and translation relations consistently; it is not persistent contract state.

6 Scalar Cost Semantics and Static Resource Analysis

6.1 Dynamic source cost

The scalar source cost is

$$C = \langle o, a, h, p, r_b, w_b, q, d \rangle \in \mathbb{N}^8.$$

o counts abstract scalar and control operations; a logical local-object allocation events; h cumulative capacity bytes reserved by those events; p peak simultaneously live local scratch bytes; r_b, w_b semantic bytes read and written; q conditional, bounds, and loop decisions; and d maximum active DSL call depth. Invoking the exported entry establishes the initial active depth $d = 1$ but is part of the ABI boundary and adds no internal call/return event. Only an explicit $f(e_1, \dots, e_k)$ expression adds $\chi_{\text{callReturn}}$ and increases active depth. Cumulative reservation h is never used as a synonym for peak storage p .

For a rooted subderivation with continuation roots K , peak scratch is the maximum, over its semantic configurations, of the region bytes in

$$\text{Reach}_{\sigma_t}(K \cup V_t) \setminus \text{Reach}_{\sigma_t}(K),$$

where V_t contains the environment values, partial literal/builder, accumulator, operands, and result that the configuration requires. Thus caller-owned roots are not charged again, but objects retained from a bound expression are charged while they remain needed. Sequential dynamic composition adds o, a, h, r_b, w_b, q and takes max for d . Its p component is the maximum over the entire enclosing trace under the enclosing rule's original K ; it cannot in general be obtained by taking only the maximum of premise peaks after earlier operands have become premise continuation roots. We

write this trace-aware composition $\oplus$. Branch join $\sqcup$ is componentwise maximum. Both operations are monotone under componentwise order $\preceq$ when their retained-root parameters are fixed.

The source model is scalar. It contains no SIMD component because `BOUNDFIN` contains no vector operation. Optional Wasm SIMD events are target events defined only in sections 8 and 9.

Semantic reads/writes refer to the abstract object representation. They exclude cache-line transfers, write allocation, prefetching, spills, speculation, and runtime metadata. The latter are empirical target quantities. An allocation reserves $\text{region}(\tau, N)$ capacity bytes even when runtime length is smaller; initialisation writes only the header and logical element representations. Complete event constants for every primitive and construct appear in tables 5 and 10.

6.2 Symbolic size and footprint summaries

Resource expressions extend the upper terms of section 4:

$$b ::= u \mid b + b \mid kb \mid \max(b, b) \mid \sum_{i=0}^{u-1} b(i) \mid \text{rec}(u, b_0, i.X.b_s),$$

where $k \in \mathbb{N}$ and the final form denotes the finite sequence $X_0 = b_0$, $X_{i+1} = b_s[i, X_i]$, returning $X_{\llbracket u \rrbracket}$. The checker accepts a sum or recurrence only when its upper term has a certificate $0 \leq u \leq N$ for a literal capacity N . Evaluation $\llbracket b \rrbracket_\eta$ is therefore a total natural-number function for every $\eta \models \Delta$. Exact component $\star$ is not a term: a rule that loses exactness must use its certified upper u , and capacity fallback uses N . Calls substitute actual terms simultaneously into an earlier declaration's finite expression; cyclic summary equations are syntactically impossible.

Peak composition requires more than a componentwise cost vector. The static judgment therefore returns

$$\Sigma; \Delta; \Gamma; \Lambda \vdash e : \tau, \kappa \triangleright \Phi_e = \langle \hat{E}_e, \hat{L}_e, \hat{P}_e \rangle. \quad (\text{A-Judgment})$$

$\hat{E}_e = \langle \hat{o}, \hat{a}, \hat{h}, \hat{r}_b, \hat{w}_b, \hat{q}, \hat{d} \rangle$ bounds cumulative events and depth. $\hat{L}_e$ bounds local bytes reachable from a successful result but not from continuation roots. $\hat{P}_e$ bounds the maximum such bytes at any point. Λ maps aggregate variables to conservative reachable-footprint expressions and is evaluated as a monotone transformer. The extracted cost bound is

$$\text{cost}(\Phi_e) = \langle \hat{o}, \hat{a}, \hat{h}, \hat{P}_e, \hat{r}_b, \hat{w}_b, \hat{q}, \hat{d} \rangle.$$

This auxiliary summary handles returned aggregates, accumulator replacement, and nested calls without pretending that peak bytes are additive.

For a type/shape pair, $\text{capfoot}(\tau, \kappa)$ is the candidate expression for the reachable graph of a value: scalars contribute zero local-object bytes; products sum component footprints with duplicate references conservatively counted; an array contributes its own capacity region plus the candidate reachable footprints of logical elements. The alias-overcount lemma in section 5 shows that duplicate references cannot make this structural sum smaller than a conforming graph. Establishing semantic conformance of substituted call results remains part of `PO-SIZE-SUBST` and `PO-COST-SOUND`.

6.3 Composition rules

A `let` first analyses e_1 , then analyses e_2 under $\Lambda[x \mapsto \hat{L}_1]$. Its event components compose sequentially, its result retention is $\hat{L}_2$, and

$$\hat{P}_{\text{let}} = \max(\hat{P}_1, \hat{P}_2). \quad (\text{C-Let-Peak})$$

$\hat{P}_2$ already includes the bound value whenever it is reachable from the body environment. A liveness refinement may remove x from Λ after its last use, but only with a checked free-variable/alias

certificate; otherwise the analysis retains it. Adequacy of this unrefined fallback is included in PO-LIVE and PO-COST-SOUND.

A conditional composes the guard events with the event join of its branches, joins result footprints, and takes the maximum of guard and branch peaks. Array literals use the separate C-Lit finite-prefix equations in section A.7: they analyse the distinct summaries $\Phi_0, \dots, \Phi_{m-1}$, add one reservation, one write after each successful element, no loop tests, and one seal only on full success. Indexing and length retain no new local object; returning an aggregate element uses its shape footprint, which may alias the input and is conservatively bounded by Λ .

For a builder, let $B \leq N$ be the exact count expression when available and otherwise the proved upper count, let $R_N = \text{region}(\tau, N)$, and let (L_j, P_j, E_j) be the body summary at index j . Define the partial reachable capacity bound

$$A_0 = R_N, \quad A_{j+1} = A_j + L_j.$$

The builder summary satisfies

$$\begin{aligned} \hat{L}_{\text{build}} &= A_B, \\ \hat{P}_{\text{build}} &= \max\left(P_n, A_B, \max_{0 \leq j < B} P_j[\Lambda(\alpha) \mapsto A_j]\right), \\ \hat{E}_{\text{build}} &= E_n \oplus_E \chi_{\text{reserve}}(R_N) \oplus_E \bigoplus_{j=0}^{B-1} (E_j \oplus_E \chi_{\text{elementWrite}, \tau}) \oplus_E \chi_{\text{seal}}. \end{aligned} \quad (\text{C-Build})$$

Here $\oplus_E$ is event composition without peak, and the body transformer includes the partial builder as a live root. If the body result is scalar, every $L_j = 0$, so the peak is one capacity region plus body temporaries. Cumulative reservation includes R_N and every nested allocation performed by body evaluations; completed writes remain counted on an error path.

For a fold, let B again denote the exact count when available and the proved upper otherwise. Let L_0 be the successful accumulator footprint and define

$$L_{j+1} = L_b(j, L_j), \quad 0 \leq j < B,$$

by instantiating the body transformer with accumulator footprint L_j . The fold returns L_B and has peak

$$\hat{P}_{\text{fold}} = \max\left(P_n, P_0, \max_{0 \leq j < B} P_b(j, L_j)\right). \quad (\text{C-Fold-Peak})$$

Its event summary composes count and initial costs, B loop-test/body summaries, and the final loop test. This recurrence permits a body result to retain its previous accumulator; in that case L_j grows and the peak reflects the retained chain. When a liveness certificate proves accumulator separation, as for the binomial row builder, the old L_j is absent from L_{j+1} and two scratch rows suffice.

Function summary $\Phi_f(\vec{n}, \vec{\lambda})$ is parameterised by formal exact/upper array lengths and caller-visible argument footprints. At an explicit call, actual size expressions and footprints are substituted simultaneously. Argument event summaries compose left-to-right; the callee summary is instantiated; $\chi_{\text{callReturn}}$ is added; and callee-relative depth is lifted by one. Aggregate returns use caller-owned result storage in the target, but their source reachable footprint remains in $\hat{L}$. The exported-entry wrapper performs only the initial depth-one lift and adds no call/return event.

6.4 Resource-soundness obligation

Proof Obligation 6.1 (Static scalar resource-bound soundness). *Let $\vdash \Sigma$ wf and suppose the typing, size, and analysis judgment derives Φ_e . For every $\eta \models \Delta$ and conforming rooted state, if*

$$\Sigma; K \vdash \langle e, \rho, \sigma \rangle \Downarrow \langle r, \sigma' \rangle \mid C,$$

then

$$C \preceq \llbracket \text{cost}(\Phi_e) \rrbracket_\eta,$$

and, on success, result-reachable local bytes are at most $\llbracket \hat{L}_e \rrbracket_\eta$.

The proof attempt is a simultaneous induction over typing, size, analysis, and evaluation derivations, strengthened with the result-footprint invariant.

Primitives and errors. The total δ_p table determines a unique success or declared error. Each dynamic terminal event is below its exhaustive table entry; generic strict propagation bounds precisely the evaluated prefix (PO-COST-PRIM and PO-COST-ERR).

Bindings and branches. Rooted liveness gives the premise used by C-Let-Peak; the branch selected dynamically is below the static join. Store reduction cannot increase reachable bytes.

Literals. Induct on remaining suffix length $m - j$. At prefix j , the fresh region and completed element graphs are conservatively bounded by A_j ; the distinct body hypothesis for e_j bounds its trace while that prefix and suffix environment are retained. Success adds exactly one write before the induction hypothesis; error stops before that write and seal; the base adds one seal. C-Lit therefore enumerates every success and error prefix without adding a loop decision (PO-COST-LIT).

Builder. Induct on the initialised prefix. At step j , the live partial region and retained elements are bounded by A_j ; the body hypothesis bounds its additional objects and writes; sealing establishes result footprint A_n . Since $n \leq u \leq N$, the bounded maximum and sum dominate every success and early-error prefix (PO-COST-BUILD).

Fold. Induct on the executed count. The simultaneous footprint hypothesis establishes L_{j+1} from L_j ; the bounded maximum dominates both a retained accumulator chain and a certified replacement. Event sums stop dynamically on error and are intended to be dominated by the full bounded prefix (PO-COST-FOLD).

Calls. Well-founded induction on $\prec_M$ validates the stored parametric summary after simultaneous size/footprint substitution. Depth lifting accounts for one active activation; caller/callee live bytes are present in the instantiated transformer (PO-COST-CALL).

For o, a, h, r_b, w_b, q , terminal-row induction and nonnegativity reduce every success or declared-error derivation to a finite prefix; literal, builder, and fold inductions identify which writes, seals, and loop tests are present. For d , a call-free entry has depth one and each explicit-call premise applies one lift. The unresolved component is p , together with the successful result footprint needed to compose it through calls. The exact missing lemma states that simultaneous substitution of an earlier declaration's result/retained-footprint transformer dominates the union of caller roots, aliased actual graphs, callee temporaries, and a returned aggregate at every success and error boundary. Without it, the nested-call case of PO-COST-ERR and PO-COST-CALL cannot establish the enclosing peak. This is missing proof exposition rather than a known counterexample to the corrected rules; no smaller counterexample shape is presently known. Definition 6.1 therefore remains open.

Theorem 6.2 (Finite capacity instantiation). *For a closed well-formed entry point, replacing every unavailable exact or upper length by its declared capacity and evaluating the accepted resource expressions yields a finite natural number for every scalar source component.*

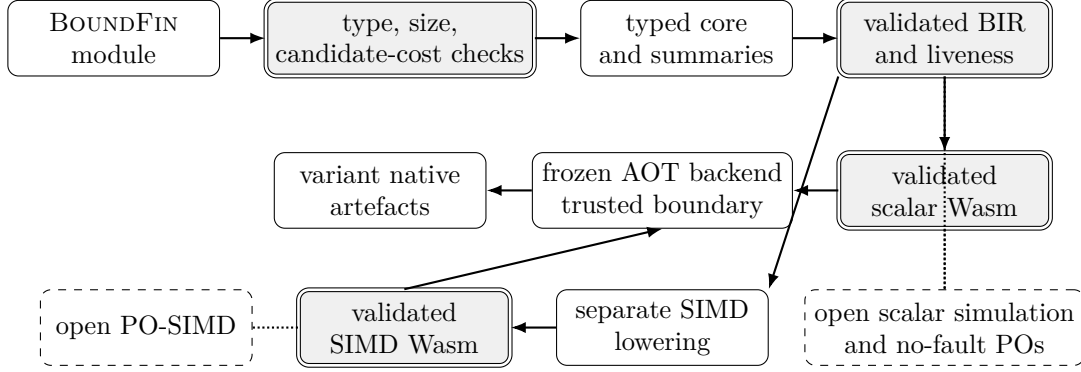


Figure 1: Specification and assurance boundaries. Double solid borders denote statically checked artefacts, dashed borders denote open proof obligations, and single solid borders denote representations, transformations, or trusted stages. The separate SIMD path is named explicitly; no distinction depends on colour.

Proof. Proceed in declaration order. Each literal capacity is at most $2^{31} - 1$ and hence finite. A resource expression for the first declaration is a finite syntax tree whose leaves are naturals or capacities; addition, nonnegative scaling, and maximum preserve finiteness, while each sum or recurrence performs at most its finite capacity many finite steps. For declaration j , every call summary refers only to an earlier declaration, so the induction hypothesis makes simultaneous capacity substitution finite. The entry summary is therefore finite componentwise. This result concerns definedness of the candidate expression only; it does not establish definition 6.1. $\square$

6.5 Interpretation

The analysis computes finite candidate source event and scratch expressions relative to the formal object model; definition 6.1 is the unresolved claim that they are upper bounds. It does not bound Wasm instructions, native frame spills, cache traffic, cycles, or latency. A compiler can erase events or introduce copies, checks, and address arithmetic. Quantitative preservation therefore requires a separate target trace and lowering envelope, just as verified stack-bound work carries a quantitative property through compilation [5]. Dynamic conformance tests may detect counterexamples and estimate tightness, but zero detected violations is not a proof of definition 6.1.

7 AOT-Wasm Translation Specification

This section specifies, but does not implement, a compiler architecture. The scalar path is the subject of the preservation target. SIMD is a separately identified target transformation and never changes the source cost judgment.

7.1 Front end and normalisation

The front end performs parsing, name resolution, primitive-table lookup, type/effect checking, exact/upper/capacity size analysis, proof checking for bounded counts, call-order validation, and module-memory admissibility. Each accepted expression carries its type, size shape, source location, scalar source summary, and footprint transformer. Rejected text has no core representation.

Proof-oriented normalisation makes source order explicit only through typed administrative bindings and explicit control-flow structure. It preserves literals, conditionals, folds, and builders as distinct constructs. It does not constant-fold, eliminate, duplicate, merge, hoist, sink, or share a source operation, even when that operation is pure, and it does not reassociate binary64 expressions. Consequently the scalar source event trace and rooted live-object trace are unchanged up to zero-event administrative steps. Target optimisation performed later by the Wasm/AOT toolchain may erase or transform work, but it is outside PO-NORM and does not retroactively alter the previously computed source candidate bound.

7.2 BIR states and well-formedness

The intermediate language BIR is a typed, first-order, SSA-like control-flow graph. Its operands, instructions, terminators, blocks, functions, modules, call-stack states, initial/final configurations, and small-step rules are defined in section B.1. Determinism of that transition relation is conditional on a validated function and the functional scalar primitive table.

Every memory operation carries a region capability

$$\text{cap} \in \{\text{inputRead}, \text{freshInit}, \text{scratchInit}, \text{outputWrite}\}.$$

An `inputRead` region is immutable. `freshInit` may write each builder header/element position once before a seal operation and never afterward. `scratchInit` is private compiler storage with no related live source object; it cannot be used to overwrite bytes in the current memory injection. `outputWrite` is caller-owned result storage and becomes observable only on successful status. There is no capability for arbitrary mutation of a live immutable source array.

A well-formed BIR function has dominated typed definitions, consistent block arguments, one terminator per block, direct rank-decreasing calls, explicit status paths, and checked regions. Each fold has a loop certificate (N, n, k) proving $0 \leq k \leq n \leq N$ and one monotone increment. Each builder has a region, initialised-prefix value j , proof $0 \leq j \leq n \leq N$, unique writes below j , and a seal transition only at $j = n$. The validation judgments are in section B.1.

7.3 Liveness, frames, and reuse

At every normalised point q , $\text{Live}(q)$ is the set reachable from its environment, continuation, accumulator, literal/builder prefix, actual arguments, and pending result. A certificate assigns each local object α a slot $(o_\alpha, z_\alpha, A_\alpha)$ and proves capacity/alignment plus

$$\alpha \neq \beta \wedge \text{LiveAt}(\alpha, q) \wedge \text{LiveAt}(\beta, q) \implies [o_\alpha, o_\alpha + z_\alpha) \cap [o_\beta, o_\beta + z_\beta) = \emptyset.$$

It also supplies the derivation $\text{Roots}(q) \cap \text{Reach}(\alpha) = \emptyset$ before reuse. The finite checking judgment is in section B.3. If checking fails, an injective no-reuse layout assigns every reservation a distinct aligned slot; this fallback is always admissible when its total extent fits the 32-bit module bound.

At semantic point t , the source-to-target placement partitions live local objects into frame-mapped $L_F(t)$ and caller-output-mapped $L_O(t)$. Define

$$P_F = \max_t \sum_{\alpha \in L_F(t)} \text{region}(\alpha), \quad P_O = \max_t \sum_{\alpha \in L_O(t)} \text{region}(\alpha), \quad P_S = \max_t (F_t + O_t).$$

P_S is the source peak p ; in general it is no greater than $P_F + P_O$. Aggregate results constructed directly in caller output are charged in O_t , not again in the callee frame. Let F_f be the aligned extent of all slots and padding assigned to one activation of f , excluding input and output regions. The

checked placement must show $F_t \leq F_f$ for points in that activation. With acyclic calls, maximum stacked frame bytes are

$$\text{Stack}(f) = F_f + \max_{g \in \text{calls}(f)} \text{Stack}(g), \quad \max \emptyset = 0. \quad (1)$$

Aggregate call results are constructed in a destination region reserved by the caller; no pointer into a returned callee frame escapes. For every entry, the checked ABI output extent must dominate O_t on every path. Thus the source local peak is explained by frame and output placements without counting the same aggregate in both.

7.4 Lowering obligations

Scalar primitives map to the exact-width BIR operation or an explicit declared-error edge. A source conditional maps to one conditional terminator and a join with typed block parameters. Products use the fixed representation table. General indexing loads the runtime length, proves/branches on bounds, and computes an offset only on the successful edge. A static index proof may remove the dynamic branch only when it dominates the access.

A literal lowers by one reservation followed by the distinct fragments $G_0, \dots, G_{m-1}$ in source order, one initialisation-only write after each successful fragment, and one seal; it has no loop header, index test, or repeated body. A numeric fold lowers to preheader, header, body, latch, and exit blocks. Header parameters carry k , n , and accumulator representation. The back edge is permitted only with the loop certificate and the checked $k + 1 \leq n \leq N$. An immutable builder separately reserves its assigned fresh region, repeats its single indexed body fragment in increasing index order, performs one initialisation-only write per success, seals the header at n , and exposes the resulting pointer only after sealing. Early error paths expose neither kind of partial object.

Direct calls evaluate actuals left-to-right, pass immutable pointers and caller-owned aggregate-result destinations, and propagate the first nonzero status. A callee cannot write an input capability. These obligations cover array literal, builder, fold, call, and returned aggregate cases separately in the traceability matrix of section B.4.

7.5 Wasm memory and ABI admissibility

The primary target uses one non-growing Wasm 32-bit linear memory. Let $U = 2^{32}$ and let $\text{align}(b, A)$ be checked alignment to power-of-two A . For every region with base b and capacity representation size z , the validator proves in mathematical integers

$$0 \leq b < U, \quad b + z \leq U,$$

before encoding the address as an i32 bit vector. The array header is two little-endian 32-bit fields (length and capacity), followed by the aligned element area. All products, offsets, $H + Nw(\tau)$ terms, alignments, and page rounding are checked for overflow before lowering.

Let $b_I = 0$, $e_I = \text{align}(b_I + \text{ABIIn}, A)$, $b_O = e_I$, $e_O = \text{align}(b_O + \text{ABIOut}, A)$, $b_F = e_O$, $e_F = \text{align}(b_F + \text{Stack}(f_{\text{entry}}), A)$, and $b_R = e_F$. Module admissibility requires

$$\text{MemReq} = \text{align}(b_R + \text{runtimeReserve}, 65536) \leq U. \quad (2)$$

Each named interval must also satisfy its own checked endpoint and all required disjointness predicates. ABIIn and ABIOut include internal layout padding; Stack includes frame-alignment gaps; runtimeReserve includes only runtime metadata not already counted. Wasm operand-stack storage

held outside linear memory is recorded as an engine resource rather than added to (2). These categories make aggregate output part of `ABIOut`, not a second frame allocation. Kernel-specific expressions such as $n + 1$ require $n \leq N_{\max} - 1$.

The module exports one contract entry and, only if the ABI requires it, memory. The entry receives input/output offsets and returns an i32 status. Status zero denotes success; nonzero values injectively encode declared errors. The core module has no imports, start function, memory growth, table, indirect call, reference type, shared memory, thread, atomic, exception, or callback. Source errors use status edges, never Wasm traps.

7.6 Control and numeric lowering

BIR blocks are structured into Wasm `block`, `loop`, `if`, and `br`. The structuring relation must neither duplicate nor reorder a primitive, call, builder write, or error edge. Scalar integer arithmetic and comparisons map to the matching Wasm operations in table 5; checked signed division is guarded before the potentially trapping target instruction. Binary64 mappings forbid relaxed operations, reassociation, reciprocal approximation, excess precision, and contraction.

The native C++ oracle/baseline must use fixed-width formulations without signed-overflow undefined behaviour and the same checked division and binary64 order. Transcendentals such as `pow` and host `libm` calls are excluded from the formal core and confirmatory kernels.

7.7 AOT and target SIMD

Validated scalar Wasm is precompiled by a frozen AOT engine before timed execution. Compilation, instantiation, first call, and steady-state invocation are distinct metrics. The engine, backend, version, target features, validator, and flags are immutable experimental factors.

Optional SIMD lowering starts from validated BIR and yields a separate module/configuration for K1 or K2 only. Eligibility, lane grouping, masks, remainder path, and reduction order are explicit. Its target event trace may count vector instructions and lanes, but these counts are not inserted into the scalar source cost. SIMD equivalence is PO-SIMD and its performance is RQ5; K3 remains MS/PO only, and failure of that obligation does not alter the scalar language definition.

8 Semantics-Preservation Verification Goals

The intended scalar-translation property is equality of observable outcomes, not equality of source cost, target events, native instructions, or elapsed time. None of the simulation statements in this section is presented as a theorem or lemma: their construct cases and fault-exclusion argument remain open.

8.1 Observations and failure classes

A source invocation outcome is

$$o_S ::= \text{abiFail} \mid \text{ok}(v, \sigma) \mid \text{err}(\epsilon), \quad \epsilon \in \{\text{Bounds}, \text{DivZero}, \text{DivOverflow}\}.$$

`abiFail` denotes `InvalidABI` before source entry. A Wasm-side outcome is

$$o_W ::= \text{ok}_W(m) \mid \text{status}(c, m) \mid \text{trap} \mid \text{exhaust} \mid \text{hostFail},$$

where a nonzero status code injectively names a declared error. Backend miscompilation and operating-system/hardware failure are execution-platform failures rather than outcomes of the validated Wasm semantics; they are reported separately in the experiment.

Function $\mathcal{O}_S$ ABI-serialises a successful rooted value graph and maps the three internal errors to their status names. Function $\mathcal{O}_W$ decodes output bytes only for zero status and otherwise decodes the injective declared-error status. Relation $\approx_{\text{obs}}$ holds when the error names agree, or successful values agree recursively: Boolean and integer words agree, products agree fieldwise, arrays have equal logical length and elementwise agreement, non-NaN binary64 words agree exactly (including signed zero), and any two NaN words agree. Thus a canonical source NaN may be related to any NaN word permitted by the fixed scalar Wasm semantics. No `trap`, `exhaust`, or `hostFail` outcome is related to a source value or declared error.

8.2 Source–BIR and BIR–Wasm state relations

A partial injection μ maps each live source object to a disjoint target region. Relation $R_{SI}(K, e, \rho, \sigma; S_I, \mu)$ holds when scalar words and flattened products follow the representation function, every object reachable from the continuation, environment, accumulator, partial literal/builder construction, or pending result lies in $\text{dom}(\mu)$, each mapped header and logical element agrees, and mapped regions of simultaneously live objects are disjoint. Bytes beyond logical length are unrelated. Literal and builder prefixes use the administrative prefix relation of section B.1 and become ordinary immutable arrays only after sealing.

At a semantic death point, μ may shrink by removing objects absent from every root. A later object may use the same region only if the checked liveness judgment of section B.3 accepts the removal before the new mapping. The no-reuse fallback assigns an injective region for the whole activation and needs no death certificate.

Relation $R_{IW}(S_I, S_W, \zeta)$ maps BIR values to Wasm locals/stack positions, BIR labels to structured continuation depths, frame and ABI regions to linear-memory intervals, status values to words, and each live capability to its permitted interval. The map ζ includes the current call-stack correspondence. Initial and final clauses, administrative stuttering, and the supported Wasm instruction relation are defined in sections B.1 and B.4.

8.3 Open simulation and representation obligations

Proof Obligation 8.1 (PO-NORM: normalisation). *Typed normalisation preserves source outcome, first-error order, scalar source event trace, size shape, and rooted live-object trace up to zero-event administrative bindings and fresh-object renaming.*

The pass introduces only typed administrative bindings and explicit control structure. It performs no constant folding, dead-operation elimination, duplication, merging, sharing, motion, or reassociation of a source operation. The structural proof must treat the distinct literal vector, builder reservation order, fold counts, and declared-root elimination explicitly. Optimisation by later target/AOT stages is outside this source-trace obligation.

Proof Obligation 8.2 (PO-SI-EXPR: source-to-BIR simulation). *If a source expression and a BIR fragment start in an R_{SI} -related state, source success is matched by reaching the fragment’s success continuation with a related value and rooted store, while the first declared source error is matched by its corresponding error continuation.*

The attempted induction covers the constructor rules in section B.4: suffix induction handles the distinct literal fragments, count/body induction handles fold and builder, and declaration-

rank induction handles calls. The unclosed dependencies are semantic size-summary substitution, aggregate-result placement, and preservation of the liveness injection across nested failure paths.

Proof Obligation 8.3 (PO-MEM: representation, death, and reuse). *Array construction, length/index operations, aggregate return placement, root removal, and any accepted slot colouring preserve representation and disjointness. Each related in-bounds source access has a related in-region target address; a source bounds error takes a status edge.*

Checked natural-number endpoints and capabilities settle individual address calculations. The open part is the global injection-update argument for products or arrays that alias an older object across a call or builder failure.

Proof Obligation 8.4 (PO-CTRL: bounded control). *A literal takes its m distinct fragments exactly once in order with m writes and no loop test; a fold of count n takes exactly n related body paths; a builder performs exactly n related repetitions and initialisation writes before one seal; and structured Wasm lowering neither duplicates nor omits a nonadministrative BIR action.*

Residual-count induction settles the source/BIR loop shape once the lowering constructor is assumed. The open CFG part requires a well-founded measure for administrative Wasm branches and a proof for every structured-control constructor.

Proof Obligation 8.5 (PO-IW: BIR-to-Wasm simulation). *From R_{IW} -related validated states, every BIR step is matched by a finite nonempty Wasm sequence to a related state, the same successful return, the same declared-error status, or an explicitly classified target fault.*

The obligation is specific to the scalar numeric mapping, structured control, capabilities, and 32-bit addresses in this design. Existing Wasm mechanisations supply the target semantics, not this compiler argument [12, 34, 35].

Proof Obligation 8.6 (PO-NOFAULT: target-fault exclusion). *For a well-formed module, ABI-valid initial memory, accepted layout and liveness certificates, and sufficient configured call stack/linear memory, every Wasm execution related to a source derivation avoids trap, exhaust, and hostFail.*

The required cases are guarded signed division, bounds-before-address order, aligned endpoints, initialised-before-read, sealed immutability, structured branch depth, finite active frames, absence of imports and memory growth, and host adherence to the closed-module ABI. Validator success alone does not prove these dynamic facts. AOT-backend, OS, and hardware failures are outside this obligation and remain named trust/experimental boundaries.

8.4 End-to-end verification goal

Proof Obligation 8.7 (PO-SEM-PRES: source-to-Wasm observable preservation). *Let M be well formed, let its scalar translation relation produce validated module W , and let the input memory be ABI valid. If the source entry evaluates to os , then the Wasm export terminates with an outcome ow such that $\mathcal{O}_S(os) \approx_{\text{obs}} \mathcal{O}_W(ow)$; in particular, ow is neither a target fault nor an undeclared status.*

The proposed composition first uses PO-NORM, then PO-SI-EXPR and PO-MEM/PO-CTRL to obtain a finite BIR path, then PO-IW to obtain a Wasm path. PO-NOFAULT is essential: without it, PO-IW's target-fault alternative prevents the displayed conclusion. The observation clauses

then yield the same declared error or recursively equivalent successful output. Since PO-NORM, PO-SI-EXPR, PO-MEM, PO-CTRL, PO-IW, and PO-NOFAULT remain open, this composition is a verification plan rather than a preservation result.

Compiling validated Wasm to native code adds another trust boundary. Verified compilation and modular instruction-selection checking can reduce that boundary only for the exact covered pass, version, operation, and ISA [21, 31]. Translation validation can provide artefact-specific evidence but is not a universal proof [22]. The native C++ oracle must canonicalise every floating primitive result before subsequent source-level use; comparison at the external boundary then applies the same NaN quotient.

8.5 Target-event envelope

Functional preservation would not imply cost preservation. A future scalar certificate may define a target event vector E_W , nonnegative matrix A , and module constant b_M satisfying

$$E_W \preceq A \llbracket \text{cost}(\Phi) \rrbracket_\eta + b_M. \quad (3)$$

This is PO-COST-TRANS. Optional SIMD lowering has its own vector and certificate; no SIMD count is projected back into the scalar source cost. Even a discharged envelope would concern abstract Wasm events, not native instructions, cache misses, cycles, or latency.

9 Experimental Methodology

This section is a preregisterable design, not an empirical report. Formal proof-obligation status is reported independently from observed conformance; the reproducibility artefact contains the more detailed implementation hand-off.

9.1 Questions, correctness gates, and estimands

RQ1

Within the declared conformance/test domain, are any counterexamples found to the instantiated scalar source bounds, and what are component-wise slack and tightness?

RQ2

Within the differential suite, are any observable mismatches found among reference semantics, BIR, scalar Wasm, and the semantically constrained C++ oracle?

RQ3

For each preregistered kernel-size family, what are the AOT-Wasm/native ratios under (a) matched scalar and (b) production-optimised configurations?

RQ4

Within each kernel, how does its own bounded workload-size factor interact with execution form, and what generated-code differences accompany that interaction?

RQ5

For the preregistered K1 and K2 families, what are the explicit Wasm and native SIMD speedups over their matched scalar forms, and what is the interaction between those speedups?

G1 and G2 are correctness gates, not hypotheses that testing can prove:

- **G1 (bound-counterexample gate):** report either a reproducible component violation, no counterexample detected in the declared domain, or unresolved coverage/instrumentation. Definition 6.1 remains an open proof obligation in every case.
- **G2 (translation-counterexample gate):** report either a reproducible $\approx_{\text{obs}}$ mismatch, no mismatch detected in the declared differential suite, or unresolved oracle/coverage. Definitions 8.6 and 8.7 remain open proof obligations.

A G1/G2 counterexample blocks affected performance configurations and reopens the corresponding formal obligation. No detected counterexample is evidence about the tested domain only.

Let $y_{spbricj}$ be primary per-contract latency in session s , paired process p , randomised block b , run r , invocation i , execution form $c \in \{W, C\}$, and family $j = (k, n, z)$. Define the run statistic $m_{spbricj} = \text{median}_i(y_{spbricj})$ and the process-condition statistic $T_{spcj} = \text{median}_{b,r}(m_{spbricj})$, giving equal weight to scheduled runs rather than to their invocation counts. The paired log contrast is

$$D_{spj}^x = \log T_{spWj}^x - \log T_{spCj}^x, \quad x \in \{\text{MS}, \text{PO}\}.$$

The population estimand is $R_j^x = \exp\{\mathbb{E}_s[\mathbb{E}_{p|s}(D_{spj}^x)]\}$, and its estimator gives sessions equal weight:

$$\hat{R}_j^x = \exp\left\{\frac{1}{S} \sum_{s=1}^S \frac{1}{P_s} \sum_{p=1}^{P_s} D_{spj}^x\right\}.$$

This is a geometric mean of paired ratios of process-level medians; it is neither a ratio of pooled medians nor a median of invocation-level ratios.

Let $\delta_L = 0.95$ and $\delta_U = 1.05$ be provisional practical-equivalence limits, frozen before confirmatory unblinding. Simultaneous 95% intervals classify each R_j as faster ($U < \delta_L$), practically equivalent ($L \geq \delta_L$ and $U \leq \delta_U$), slower ($L > \delta_U$), or unresolved. MS and PO form distinct interval families and are not pooled.

RQ4 estimates execution-form by size contrasts separately for each k . Its null region is the same frozen band on ratio-of-ratios between adjacent preregistered size levels or a kernel-specific log-size slope contrast. K1 depth, K2 record count, and K3 record count are not treated as the same numerical scale. A secondary cross-kernel analysis is permitted only after preregistering a normalised source quantity such as inferred operations, semantic bytes, or decisions; kernel-specific results remain primary.

For RQ5, j ranges only over K1 and K2. Replace T_{spcj} in the same paired/session-weighted construction and define

$$S_j^W = T_j^{W, \text{MS}} / T_j^{W, \text{SIMD}}, \quad S_j^C = T_j^{C, \text{MS}} / T_j^{C, \text{SIMD}}, \quad I_j^{\text{SIMD}} = S_j^W / S_j^C.$$

Intervals and the frozen band classify each speedup and interaction. For K1 SIMD, however, the raw timed unit is a full two-contract batch: if $B_{\text{mathrm K1}, spbrj}^{X, \text{mathsf{SIMD}}}$ is its elapsed time under execution form $X \in \{W, C\}$, then each observation entering the run statistic is

$$y_{\text{mathrm K1}, spbrj}^{X, \text{SIMD}} = B_{\text{mathrm K1}, spbrj}^{X, \text{SIMD}} / 2.$$

The full batch includes packing, the actual call boundary, both computations, unpacking, both statuses, and symmetric consumption of both results. This derived quantity is an *amortised per-contract batch time*, never a single-contract latency; full-batch time and contracts per second are retained as secondary quantities. Disassembly must confirm the intended vector transformation, and performance and numerical-order outcomes are reported separately.

9.2 Three non-pooled build matrices

The exact stable AOT runtime/backend, validator, Clang release, target triple/CPU/features, linker, standard library, and feature sets are frozen before data collection. A toolchain change defines a new experiment.

Matched scalar (MS). Both paths use comparable O3/interprocedural policies, strict source numerics, and disabled loop/SLP auto-vectorisation. The native formulation avoids undefined signed overflow and implements the source checks. Disassembly must show no kernel vector instruction. This comparison isolates scalar code-generation, sandbox, and call-boundary effects.

Production-optimised (PO). Both paths use their normal production O3/LTO and target-feature policies, including legal auto-vectorisation and architecture features, recorded verbatim. Fast-math, unsafe reassociation, and semantic check removal remain disabled because they would change the contract. Any safe vectorisation is an observed code-generation outcome. This comparison is reported separately from MS and is the production-relevant native baseline requested by RQ3.

Explicit SIMD. Eligible K1 and K2 BIR patterns lower to an explicit Wasm SIMD profile and to a matched native explicit-vector formulation with fixed lanes, masks, remainder path, and reduction order. Auto-vectorisation is disabled around those explicit kernels so the intended transformation is identifiable. A variant-specific oracle is used where reduction order differs from scalar. K3 has no explicit-SIMD configuration: masked or predicated admission is outside RQ5, while incidental K3 auto-vectorisation remains an observed PO outcome. Explicit-SIMD results are RQ5 only and are not merged with PO.

AOT compilation precedes timed processes. Compile, validation, load/instantiation, first invocation, and steady-state invocation are recorded separately. All configurations use identical input bytes and a semantically equivalent contract boundary.

9.3 Kernels and bounded workloads

The labels compute-heavy, memory-sensitive, and branch-sensitive motivate kernel selection; they are not confirmatory factor levels and do not support claims about entire workload classes.

K1: bounded European binomial valuation. Let runtime depth $n \leq D$ with $D \leq N_{\max} - 1$. For terminal index j , define a source-expressible underlying value without **pow**:

$$P_{n,j} = \mathbf{fold}_D(j; i, s; S_0; s u) \cdot \mathbf{fold}_D(n - j; i, s; 1; s d).$$

The terminal row and backward rows are

$$\begin{aligned} A_0 &= \mathbf{build}_{D+1}(n + 1; j. \mathbf{let } p = P_{n,j} \mathbf{ in if } p > K \mathbf{ then } p - K \mathbf{ else } 0), \\ A_{k+1} &= \mathbf{build}_{D+1}(n - k; j. \beta(qA_k[j + 1] + (1 - q)A_k[j])), \quad 0 \leq k < n, \end{aligned}$$

and the value is $A_n[0]$. The second equation is the body of $\mathbf{fold}_D(n; k, A; A_0; \dots)$. At step k , $\text{len}(A_k) = n + 1 - k$; the builder count is $n - k \leq D + 1$; and $j < n - k$ implies both $j < n + 1 - k$ and $j + 1 < n + 1 - k$. Hence every displayed index is statically safe.

Let $R_D = \text{region}(\text{f64}, D + 1)$. Exactly $n + 1$ row builders allocate, so cumulative reservation is $(n + 1)R_D$. The liveness certificate proves A_k is not reachable from A_{k+1} ; peak row scratch is R_D

for $n = 0$ and $2R_D$ for $n > 0$. No in-place source update is used. Total logical element writes are

$$(n + 1) + \sum_{r=1}^n r = \frac{(n + 1)(n + 2)}{2},$$

plus one H -byte header per row. Put $B_n = n(n + 1)/2$ for backward nodes, $W_n = (n + 1)(n + 2)/2$ for initialised elements, and

$$L_n = (n + 1)(n + 2) + W_n + 2(n + 1)$$

for source loop-test events (terminal power folds, all row builders, and the outer fold). There are $2B_n + 1$ successful index reads. A conservative primitive count is

$$P_n = n(n + 1) + 3(n + 1) + 5B_n + (B_n + 2n + 2),$$

where the final parenthesis counts one $j + 1$ per backward node, $n - j$ for every terminal node, $n - k$ for every backward row, and the initial $n + 1$. The payoff subtraction is charged for every terminal node because the static branch join takes its larger primitive count. Excluding the host ABI boundary, an audited component upper bound derived from table 10 is

$$\begin{aligned} a &= n + 1, & h &= (n + 1)R_D, & r_b &= 12(2B_n + 1), \\ w_b &= 8W_n + H(n + 1), & q &= L_n + (n + 1) + (2B_n + 1), & d &= 1, \\ p &\leq \begin{cases} R_D, & n = 0, \\ 2R_D, & n > 0, \end{cases} \\ o &\leq P_n + (2B_n + 1) + L_n + W_n + 3(n + 1). \end{aligned}$$

The terms in o are, respectively, P_n primitives, $2B_n + 1$ index actions, $n + 1$ payoff selections, L_n loop tests, $n + 1$ reservations, W_n element writes, and $n + 1$ seals. The read formula charges a 4-byte length and 8-byte element for every successful index; writes charge 8 bytes per element and the 8-byte header per row. There is no source allocation other than the rows and no source call beyond the entry, so $d = 1$. The expressions are analyser cross-checks, not observations; any disagreement with the normative rules is a G1 counterexample rather than a reason to alter measured traces.

Timed depths are $n \in \{8, 16, 32, 64, 128, 256, 512\}$. Three exact-dyadic parameter strata are frozen: at-the-money $(S_0, K) = (1, 1)$, in-the-money $(5/4, 1)$, and out-of-the-money $(3/4, 1)$; all use

$$(u, d, q, \beta) = (9/8, 7/8, 1/2, 1), \quad q = \frac{\beta^{-1} - d}{u - d} = \frac{1}{2}.$$

Thus the risk-neutral relation is exact for the frozen parameters. The ordinary input β and its multiplication remain in the source expression and operation tree even though its frozen timed value is one. Correctness inputs additionally cover $n = 0, D$, signed zeros, subnormals, infinities, and NaNs. Timed intermediates must remain finite, and scalar formulations preserve the displayed order.

K1 explicit SIMD is cross-contract batching with Wasm f64x2 width $w = 2$ and the same two-lane native vector width. One full batch contains two independent contracts with the same n and stratum. Its timed region includes packing, the actual host/export call, both computations, unpacking, both statuses, and symmetric consumption of both results. RQ5 uses the full-batch time divided by two as the amortised per-contract batch time defined above; full-batch time and contracts per second are secondary. No batched quantity is labelled single-contract latency. An odd final contract uses the scalar remainder path, is reported separately, and is excluded from the full-batch estimator. Ordinary scalar single-contract latency remains the RQ3 reference.

K2: portfolio mark-to-market. Inputs are three equal-length f64 arrays (q, p, c) with common capacity N and $n \leq N$; q_i is signed quantity, $p_i > 0$ is price, and c_i is a signed per-position adjustment. The kernel is

$$V = \mathbf{fold}_N(n; i, v; 0; v + (q_i p_i + c_i)).$$

The scalar order is increasing index with operation tree $v + ((q_i p_i) + c_i)$. The sizes are

$$16, 64, 256, 1024, 4096, 16384.$$

Timed values are selected from the exactly representable set

$$\mathcal{D} = \{s m 2^e : s \in \{-1, 1\}, m \in \{1, 1 + 2^{-10}, 1 + 2^{-20}\}, -8 \leq e \leq 8\},$$

with price signs fixed positive. A dataset is built by cycling through the finite field-specific multisets and ordering elements by the 256-bit value $\text{SHA256}(\text{seed} \parallel \text{field} \parallel i)$; ties use i . In the independent stratum, field tags are distinct. In the concordant stratum, $|q|$ and p use the same exponent rank; in the discordant stratum, the p exponent rank is reversed. Signs, mantissas, and c retain independent tags. The canonical byte encoding, seed, hash implementation/version, and resulting array hash are frozen.

The structure-of-arrays layout, 8-byte fields/alignment, and no-prefetch policy are fixed. Correctness cases add empty/maximal arrays, signed zeros, subnormals, infinities, NaNs, and exponent-boundary words; timed values are finite and the stated domain prevents overflow at the maximum n . Scalar reference/Wasm/C++ results require bit equality modulo the NaN quotient. Explicit SIMD uses two lane accumulators for indices $i \bmod 2$, followed by lane-0-plus-lane-1; Wasm and native SIMD must agree bitwise under that same variant order. Scalar-SIMD numerical differences are reported separately and are never treated as an equivalence tolerance.

K3: order admission and fee aggregation. K3 is selected to expose fixed nested control and controlled branch-outcome/sequence patterns; it is not intended as a masked-vector workload. Each input record is

$$r = (\text{active:bool}, \text{buy:bool}, q:\text{f64}, p:\text{f64}, \ell:\text{f64}, q_{\max}:\text{f64}).$$

The fold state is $(c:\text{i64}, v:\text{f64}, f:\text{f64})$, initially $(0, 0, 0)$. Its body evaluates nested conditionals in this exact order: `active`; $q > 0$; $q \leq q_{\max}$; `side`; and $p \leq \ell$ for buy or $p \geq \ell$ for sell. If any admission predicate is false, the state is unchanged. Otherwise it computes $x = qp$, then rate

$$\varrho(x) = \mathbf{if } x < t_1 \mathbf{ then } r_0 \mathbf{ else (if } x < t_2 \mathbf{ then } r_1 \mathbf{ else } r_2),$$

and returns $(c+1, v+x, f+(x\varrho(x)))$ in the displayed left-to-right floating order. Fixed exact-dyadic parameters are $(t_1, t_2) = (2, 8)$ and $(r_0, r_1, r_2) = (2^{-10}, 2^{-9}, 2^{-8})$. Sizes match K2; consequently the admitted count cannot overflow i64. Timed inputs are ABI valid and finite, so K3 has no internal declared error; correctness suites exercise boundary rejection and special floating words. Scalar outputs require integer bit equality and floating equality modulo NaN.

K3 is evaluated only in MS and PO. There is no explicit-SIMD K3 variant, masked/predicated K3 is outside scope, and any legal vectorisation observed in PO is reported as compiler behaviour rather than an RQ5 treatment.

For target admission proportion $\pi \in \{0.10, 0.50, 0.90\}$, the admitted count is the nearest integer to πn with ties to even. Admitted templates cycle through buy/sell and the three fee tiers using $x \in \{1, 4, 16\}$ with exact-dyadic $q, p, \ell, q_{\max}$ that satisfy every predicate. Rejected templates are allocated as evenly as possible among inactive, nonpositive quantity, over-cap quantity, and price-test

failure; earlier fields are set to pass for a record assigned to a later failure. Hash-ranked exact-dyadic perturbations from $\mathcal{D}$ are accepted only when they preserve the assigned predicate outcomes and fee tier. The seed, template counts, rejection-site counts, and multiset hash are frozen before ordering.

Outcome count and sequence structure are separate factors. Each frozen multiset is arranged as (i) hash-ranked uniform order, (ii) maximum-transition order alternating admitted/rejected outcomes while either remains, and (iii) clustered order whose run lengths cycle through (1, 2, 4, 8) and whose initial outcome is hash-selected. The same record bytes and marginal fields are used across patterns; only order changes. Realised outcome rate, binary entropy, lag-one transition rate, and run-length summary are recorded. For every static conditional site—active, positive quantity, cap, side, price, tier 1, and tier 2—the trace records evaluations, true outcomes, false outcomes, and skips caused by an earlier decision. Admission rate alone is never called branch predictability.

Correctness-only inputs cover empty/maximal arrays, invalid indices, division errors, integer boundaries, signed zeros, subnormals, infinities, and NaNs. Timed inputs are finite. Synthetic datasets are primary and immutable; licensed real-data sensitivity sets remain separate.

9.4 Correctness and bound validation

The reference evaluator follows section A.6 independently of generated target execution. Required suites are complete primitive/rule conformance, negative diagnostics, generated well-typed modules, controlled invalid mutations, metamorphic relations with numeric preconditions, differential reference/BIR/Wasm/C++ evaluation, literal/builder prefix failures, accumulator-retention cases, liveness/frame-reuse cases, and 32-bit boundary calculations.

Every failure retains source, input bytes, seed lineage, configurations, results, traces, and minimized case. For G1, the trace records dynamic $D = \langle o, a, h, p, r_b, w_b, q, d \rangle$. Reports compare exact-parametric, symbolic-upper, and capacity-only bounds separately. A violation is $D_j > B_j$; for nonzero D_j , tightness is B_j/D_j and relative slack is $(B_j - D_j)/D_j$. Peak p is also checked against instrumented maximum reachability, BIR frame certificates, and configured linear-memory maxima.

9.5 Machine controls and invocation decomposition

The primary machine is dedicated during a session. The manifest records CPU model, stepping, microcode, cores, SMT, cache and memory topology, NUMA, firmware, kernel and OS, mitigations, governor, frequency, turbo, temperature, page and locking policy, ASLR, runtime security settings, and counter permissions. The process is pinned to one physical core; its sibling is disabled or enforced idle; code and data are on one NUMA node. Migration, throttling, configuration drift, counter failure, or predeclared competing load invalidates a run independent of its latency value.

Primary per-contract timing includes the actual host-to-export call, required ABI validation/copying, kernel, status/output handling, and symmetric result consumption. Diagnostic measurements decompose: (D0) empty timestamp/harness, (D1) host-to-Wasm boundary with a no-work export, (D2) ABI validation/copying with an identity body, (D3) kernel-body interval using target-supported internal markers or counter regions, (D4) result handling, and (D5) optional in-module repeated execution. D5 may amortise D1/D2, but is labelled diagnostic and never replaces or is pooled with the primary per-invocation result. Lumos similarly motivates separating execution from I/O/serialisation and lifecycle overhead, although its serverless boundary is broader than ours [23].

9.6 Replication, timing, and tail eligibility

The hierarchy is session $\rightarrow$ paired process $\rightarrow$ randomised block $\rightarrow$ run $\rightarrow$ batch $\rightarrow$ invocation. A run is one scheduled condition segment in one process; an invocation is one contract call within a latency stream or throughput batch; a stored sample is one invocation latency or one declared batch-level measurement. These levels are not interchangeable replications.

Before confirmatory data, an untimed-for-claims pilot uses five independent sessions, four fresh pairs per session, K1/K2/K3 at their smallest, middle, and largest sizes, and initially $L_0 = 4,096$ valid ordered observations per run. Pilot observations are permanently tagged and excluded from all confirmatory estimates. For a run of length L , let $\hat{\rho}_L(\ell)$ be the lag- ℓ sample autocorrelation and define

$$\mathcal{B}(L) = \left\{ b \in \mathbb{N} : 1 \leq b \leq \left\lfloor \frac{L}{64} \right\rfloor - 9 \wedge |\hat{\rho}_L(\ell)| < 0.1 \text{ for every } \ell = b, \dots, b+9 \right\}.$$

If $\mathcal{B}(L_0)$ is nonempty, freeze its minimum as the run’s moving-block length; the upper limit guarantees at least 64 nonoverlapping blocks even at lag $b+9$. If it is empty, extend that affected run deterministically to a total of $L_1 = 16,384$ valid observations and apply the same rule from scratch to the complete extended run. The frozen sensitivity lengths are $\max\{1, \lfloor b/2 \rfloor\}$ and $2b$. If $\mathcal{B}(L_1)$ is still empty, or if the base or either sensitivity length cannot form at least 64 nonoverlapping blocks, the affected family is unresolved and must be redesigned before confirmatory unblinding; there is no arbitrary default block length.

The pilot also determines timer overhead and empirical centred residual distributions for session, pair, and within-run components of D_{spj} . For candidates $S \in \{10, 12, \dots, 30\}$ and $P \in \{4, 6, 8, 10\}$, 10,000 fixed-seed hierarchical simulations resample those components, apply the planned simultaneous interval procedure, and estimate the probability that the largest log-scale interval half-width in each MS/PO family is at most $\log(1.05)$. The smallest (S, P) with estimated probability at least 0.90 is frozen; ties minimise SP and then prefer larger S . If no candidate qualifies, the confirmatory family is reduced and preregistered before unblinding or its precision goal is declared infeasible. No pilot variance or chosen sample count is supplied in this paper because no pilot has been run.

The confirmatory minimum remains ten independent sessions on distinct time blocks/days and four fresh paired processes per session. If fewer than eight valid sessions survive, session-level inference is unresolved and the experiment is extended under the original schedule or preregistered again; a three-cluster bootstrap is prohibited.

Within each pair, execution order is randomised and counterbalanced separately for every build family/kernel/size/stratum. Warm-up has a frozen minimum and stability rule, with failure reported. Throughput uses calibrated batches of at least 1 ms. Individual latency uses a calibrated monotonic timer and, where valid, an invariant cycle source. Empty/harness overhead is always reported; any subtraction is symmetric, preregistered, and accompanied by raw estimates. Invocation order is retained for autocorrelation and drift diagnostics.

p50, p95, and p99 require dependence-aware intervals. p99.9 additionally requires at least one million valid invocations per condition, at least 1,000 nominal observations in the upper 0.1%, at least 200 tail-containing dependence blocks across at least ten sessions, and a stable finite block-bootstrap interval under preregistered block-length sensitivity. It is omitted if effective support or dependence diagnostics fail. Raw invocation count alone is not sufficient.

9.7 Metrics and statistical analysis

Primary metrics are paired process-level median invocation ratios R_j^{MS} and R_j^{PO} . Co-primary descriptive outputs are median cycles and throughput ratios. Secondary outputs include eligible quantiles,

Table 4: Predeclared evidence categories. Empirical outcomes do not discharge a proof obligation.

Item	Formal status	Empirical outcomes	Decision consequence
G1	PO 6.1 open	Counterexample detected; no counterexample detected in declared domain; or unresolved coverage	Counterexample reopens rule/table and blocks affected performance. Absence does not discharge the obligation.
G2	POs 8.7, 8.6 open	Mismatch/fault detected; none detected in suite; or unresolved oracle/coverage	Mismatch or target fault blocks artefact. Absence does not discharge either obligation.
RQ3-MS/PO	Empirical only	Per-family faster, equivalent, slower, or unresolved from simultaneous interval and frozen band	Report both matrices separately; no pooled direction.
RQ4	Empirical within kernel	Interaction below, inside, above, or overlapping the band	Report kernel-specific size contrast; no workload-class generalisation.
RQ5 (K1/K2 only)	Empirical plus PO-SIMD	Wasm/native speedups and interaction classified; transformation present/absent; numerical pass/fail	Performance interpretation requires intended disassembly and variant-oracle correctness; K3 remains MS/PO only.

instructions, branches/misses, cache misses, documented memory-controller traffic, code/artefact size, compilation, instantiation/load, first call, source-bound slack, cumulative allocation h , peak scratch p , BIR frame/stack maxima, and total configured linear memory. Source semantic bytes are never equated with hardware traffic.

Analysis uses the log paired contrasts defined above. With at least ten valid sessions, a fixed-seed hierarchical paired bootstrap resamples sessions with replacement, then pairs within each selected session, then circular moving blocks of the pilot-frozen length within each selected run; it recomputes run medians, process contrasts, equal-session estimators, and every contrast jointly. Use $B = 10,000$ replicates. For an interval family J , let $\hat{d}_j = \log \hat{R}_j$, let s_j be the bootstrap standard deviation, and let

$$M_b = \max_{j \in J} \left| \frac{\hat{d}_j^{*(b)} - \hat{d}_j}{s_j} \right|.$$

With $c_{.95}$ the empirical 0.95 quantile of M_b , report simultaneous intervals $\exp(\hat{d}_j \pm c_{.95}s_j)$. MS, PO, and each SIMD contrast family are separate. If any $s_j = 0$, fewer than ten sessions remain, moving-block resampling cannot be formed, or interval endpoints are unstable by more than 10% across the preregistered half/double block-length sensitivity, classification is unresolved. Session-level paired sign-flip intervals and cluster-robust t intervals are labelled sensitivity analyses only and cannot replace the primary interval.

Holm correction is used only for separately labelled secondary point-null tests within each preregistered family. It is not applied to the simultaneous intervals and does not determine practical-equivalence classification. Thus multiplicity is not counted twice or conflated across procedures.

RQ4 uses one model or contrast family per kernel: execution form, the kernel’s own size encoding, and their interaction, with session and pair blocking. Kernel identity may enter a combined model only as a categorical fixed factor with kernel-specific size terms; “compute-heavy/memory-sensitive/branch-sensitive” is never a statistical factor. Model diagnostics and paired nonparametric contrasts are reported; a failed model is not silently replaced. Valid statistical outliers remain in the primary analysis.

9.8 Generated code, artefacts, and threats

Timing and counter runs use the same optimised artefact; counters are collected separately without unsupported multiplexing. Preserved disassembly records kernel boundaries, instructions, branches, loads/stores, vector widths, spills, frames, bounds checks, trampolines, and remainder paths. Differences are explanatory candidates unless their contrast was preregistered.

The artefact preserves source/configuration/tool hashes, all three build matrices, ABI/call-boundary variants, randomisation, seeds/input hashes, raw order, invalidity evidence, counters, binaries, disassembly, environment snapshots, and deterministic derived-table lineage. Raw data are append-only.

Internal threats include asymmetric ABI/call work, compiler settings, timer overhead, drift, migration, and eliminated work; matched contracts, decomposition, pairing, controls, and disassembly mitigate them. Construct threats include treating source operations as time, cumulative bytes as peak memory, or an amortised in-module loop as per-contract latency; all are reported separately. External validity is limited to three named kernels, one runtime/compiler/machine, and selected sizes. The design motivations do not license class-wide claims. Conclusion threats include dependent samples, few high-level clusters, multiple families, and weak tail support; the session minimum, simultaneous intervals, block methods, and explicit unresolved outcomes address but do not remove them.

10 Results

No compiler or experiment has been executed, so no empirical result is reported. A later version will first report G1/G2 coverage and retained counterexamples or mismatches, then separate matched-scalar and production-optimised estimates, within-kernel size contrasts, the K1/K2-only explicit-SIMD ablation, call-boundary diagnostics, and generated-code/memory evidence. K1's divided batch quantity will be labelled amortised per-contract batch time, never single-contract latency. Exact, upper, and capacity bounds and cumulative, peak, frame, output, and total-memory quantities will remain separate columns.

11 Discussion

Discussion is deferred until measurements and artefact checks exist. It will keep language termination, open static-analysis obligations, open translation obligations, and machine observations as distinct claim levels. Conclusions will be limited to K1–K3 and the measured runtime, compiler, host boundary, and machine; the kernel motivations will not be generalised into sampled workload classes.

12 Threats to Validity

The predeclared threats are asymmetric ABI work, compiler-option drift, timer overhead, dependence, session loss, weak tail support, incomplete counters, oracle defects, and synthetic-input scope. Pairing, configuration hashes, disassembly, invalid-run rules, dependence-aware intervals, correctness gates, and immutable datasets mitigate but do not remove these threats. A results-bearing version must add observed exclusions, deviations, missing data, interval diagnostics, and external-validity limits.

13 Conclusion

The present contribution is a restricted source calculus, a scalar candidate-bound and liveness analysis, a formal AOT-Wasm translation design, and a reproducible evaluation methodology for stateless financial kernels. Source termination and finite capacity instantiation are proved; resource soundness, type/size preservation, compiler simulation, target-fault exclusion, and end-to-end observable preservation remain explicit proof obligations. No statement about native-relative performance or latency follows until the planned artefact and experiments exist.

A Source Calculus and Static Judgments

This appendix is normative for the source language. Surface conveniences outside these lexical categories have no core meaning.

A.1 Lexical and abstract syntax

Identifiers match `[A-Za-z_][A-Za-z0-9_]*` and exclude displayed keywords. A Boolean literal is `true` or `false`; an integer or binary64 word literal is respectively `0x` followed by exactly 8 or 16 hexadecimal digits and carries its type annotation. A capacity is an unsigned decimal numeral denoting a mathematical $N \leq 2^{31} - 1$. Whitespace separates tokens; the core has no comments or decimal floating literals. A front end may erase comments before core parsing, but that erasure is outside this grammar.

Let x, f range over variable and function identifiers, p over the finite primitive set below, $j, k, m, N \in \mathbb{N}$, and $z_w \in \{0, 1\}^w$. The abstract syntax is

$$\begin{aligned}
 b &::= \text{bool} \mid \text{i32} \mid \text{i64} \mid \text{f64}, \\
 \tau &::= b \mid \tau_1 \times \dots \times \tau_k \mid \text{arr}(\tau, N), \\
 c &::= \text{true} \mid \text{false} \mid \text{i32bits}(z_{32}) \mid \text{i64bits}(z_{64}) \mid \text{f64bits}(z_{64}), \\
 e &::= x \mid c \mid p(e_1, \dots, e_k) \mid \text{let } x = e \text{ in } e \mid \text{if } e \text{ then } e \text{ else } e \\
 &\quad \mid (e_1, \dots, e_k) \mid \pi_j(e) \mid [e_0, \dots, e_{m-1}]_N \mid \text{len}(e) \mid e[e] \mid f(e_1, \dots, e_k) \\
 &\quad \mid \text{fold}_N(e; i, x; e; e) \mid \text{build}_N(e; i.e), \\
 D &::= f(x_1:\tau_1, \dots, x_k:\tau_k) : \tau = e, \quad M ::= D_1 \dots D_m \text{ export } f.
 \end{aligned}$$

Names bound by a declaration, let, fold, or builder are pairwise fresh up to alpha-renaming. A module is closed when every call names a declaration and the export names one declaration. Declaration D_i may call only D_j with $j < i$.

A.2 Values, stores, results, and observations

Scalar values are words with their declared interpretation; product values are finite tuples; array values are unobservable object identifiers:

$$v ::= \text{bool}(z_1) \mid \text{i32}(z_{32}) \mid \text{i64}(z_{64}) \mid \text{f64}(z_{64}) \mid (v_1, \dots, v_k) \mid \alpha.$$

An environment ρ is a finite variable-to-value map. A finite store maps identifiers either to a sealed object or an administrative literal/builder construction prefix:

$$\begin{aligned}
 \sigma(\alpha) &= \text{seal}(\tau, N, \ell, \omega, v_0, \dots, v_{\ell-1}), & 0 \leq \ell \leq N, \\
 \sigma(\alpha) &= \text{pref}(\tau, N, n, j, \omega, v_0, \dots, v_{j-1}), & 0 \leq j \leq n \leq N,
 \end{aligned}$$

where $\omega \in \{\text{abi}, \text{local}\}$. Only the literal or builder construction auxiliary can name a prefix object; ordinary source expressions can read only sealed objects. Results are $r ::= \text{ok}(v) \mid \text{err}(\epsilon)$ with internal $\epsilon \in \{\text{Bounds}, \text{DivZero}, \text{DivOverflow}\}$; `InvalidABI` is a boundary result before source evaluation.

For roots V , $\text{Reach}_\sigma(V)$ is the least identifier set closed under identifiers in values and object elements. Stores $\sigma_1 \equiv_V \sigma_2$ when a bijection between their reachable identifiers preserves type, capacity, length/prefix, origin, scalar words, tuple position, and identifier edges. For errors, returned roots are K ; for success, they are $K \cup \{v\}$. This defines the fresh-object and garbage quotient used by definition 5.1.

A.3 Primitive function

Let sgn_w interpret two's-complement, word_w reduce modulo 2^w , and `can` be the NaN canonicalisation of section 3. The following table is the entire primitive signature/function map δ_p ; no implicit conversion exists.

Integer-floating conversion, shifts, general bitwise operations, square root, min/max, fused operations, transcendentals, and host-library calls are absent. Source maximum is comparison plus conditional.

A.4 Count expressions, certificates, and refinement rules

Size terms t , upper terms u , constraints ϕ , environments η , and satisfaction are those of section 4. Certificate derivations are generated only by

$$\frac{\phi \in \Delta}{\Delta \vdash_{\text{cert}} \phi} \text{ASSUME} \quad \frac{}{\Delta \vdash_{\text{cert}} u \leq u} \text{REFL} \quad \frac{\Delta \vdash u_1 \leq u_2 \quad \Delta \vdash u_2 \leq u_3}{\Delta \vdash u_1 \leq u_3} \text{TRANS}$$

and the rules below (all omitted turnstiles are $\vdash_{\text{cert}}$):

$$\frac{a \leq b \quad c \leq d}{a + c \leq b + d} \text{ADD} \quad \frac{a \leq b}{ka \leq kb} \text{SCALE } (k \in \mathbb{N}),$$

$$\frac{}{a \leq \max(a, b)} \text{MAXL} \quad \frac{}{b \leq \max(a, b)} \text{MAXR} \quad \frac{a \leq c \quad b \leq c}{\max(a, b) \leq c} \text{MAXU}.$$

Equality is checked as two inequalities; a substitution node replaces equals in a previously checked formula. A closed arithmetic leaf is accepted exactly when direct natural-number evaluation makes it true. Every node contains its premises, so certificate checking is a terminating recursion over a finite tree.

Lemma A.1 (Certificate soundness). *If $\Delta \vdash_{\text{cert}} \phi$ and $\eta \models \Delta$, then $\eta \models \phi$.*

Proof. Induction on the finite certificate. Assumption and closed evaluation follow by definition; reflexivity, transitivity, addition, nonnegative scaling, substitution, and the three maximum rules preserve the usual order on $\mathbb{N}$. $\square$

Write $\text{NW}_+(\nu_1, \nu_2, \nu)$ for $\nu = \nu_1 + \nu_2 \wedge 0 \leq \nu \leq 2^{31} - 1$, and NW_- for $\nu = \nu_1 - \nu_2 \wedge 0 \leq \nu_2 \leq \nu_1 \leq 2^{31} - 1$. The count judgment $e \Rightarrow (\lambda, u; \nu)$ has exactly these constructors:

No other expression derives a count refinement. In particular, modular overflow, multiplication of two runtime sizes, division, floating conversion, and an unproved call summary cause rejection.

Table 5: Scalar primitive signatures and functional source semantics. The metavariable w ranges over 32 and 64. Floating arithmetic first applies the fixed scalar Wasm operation and then `can`.

Primitive	Signature	δ_p
<code>bool.not</code> ; <code>bool.and/or</code>	$\text{bool} \rightarrow \text{bool}$; $\text{bool}^2 \rightarrow \text{bool}$	Boolean negation; strict conjunction/disjunction.
<code>eq/ne</code> on Boolean	$\text{bool}^2 \rightarrow \text{bool}$	Boolean equality/inequality.
<code>neg_w</code> ; <code>add/sub/mul_w</code>	$iw \rightarrow iw$; $iw^2 \rightarrow iw$	Mathematical operation followed by <code>word_w</code> .
<code>divs_w</code>	$iw^2 \rightarrow iw$	<code>DivZero</code> for divisor zero; <code>DivOverflow</code> for $-2^{w-1}/-1$; otherwise quotient toward zero.
<code>rems_w</code>	$iw^2 \rightarrow iw$	The same two error conditions; otherwise the corresponding signed remainder.
<code>eq/ne_w</code>	$iw^2 \rightarrow \text{bool}$	Word equality/inequality.
<code>lt/le/gt/ge_{s_w}</code>	$iw^2 \rightarrow \text{bool}$	Ordered comparison of <code>sgn_w</code> values.
<code>fneg/fabs</code>	$\text{f64} \rightarrow \text{f64}$	Wasm sign operation followed by <code>can</code> .
<code>fadd/fsub/fmul/fdiv</code>	$\text{f64}^2 \rightarrow \text{f64}$	Scalar Wasm binary64 result followed by <code>can</code> ; no contraction or reassociation.
<code>feq/fne/flt/fle/fgt/fge</code>	$\text{f64}^2 \rightarrow \text{bool}$	Wasm comparison, including NaN and signed-zero behavior.

Table 6: Accepted count-refinement fragment. Every displayed entailment is a checked certificate.

Rule	Premises	Result
C-Const	literal signed value $n \in [0, 2^{31} - 1]$	$(n, n; \nu)$ with $\nu = n$
C-ABI	ABI length symbol n_x , declaration capacity N	$(n_x, N; n_x)$ and $0 \leq n_x \leq N$
C-Idx	body index i under $0 \leq i < \nu_n \leq u_n$	$(i, u_n; i)$
C-Len-E	e has <code>array</code> ($s, u, N; \bar{\kappa}$)	$(s, u; \nu)$ with $\nu = s$
C-Len-U	e has <code>array</code> ($\star, u, N; \bar{\kappa}$)	$(\star, u; \nu)$ with $0 \leq \nu \leq u$
C-If-E	guard Boolean; both branches $(s, u_t; \nu_t)$ and $(s, u_f; \nu_f)$	$(s, \max(u_t, u_f); \nu)$ with $\nu = s$
C-If-U	branch exact terms differ or either is $\star$	$(\star, \max(u_t, u_f); \nu)$ with $0 \leq \nu \leq \max(u_t, u_f)$
C-Add	operands $(\lambda_i, u_i; \nu_i)$; certificate <code>NW₊</code>	$(\lambda_1 + \lambda_2, u_1 + u_2; \nu)$ if both exact, else $(\star, u_1 + u_2; \nu)$
C-Sub	operands $(\lambda_i, u_i; \nu_i)$; certificate <code>NW₋</code>	$(\lambda_1 - \lambda_2, u_1; \nu)$ if both exact, else $(\star, u_1; \nu)$
C-Call	earlier summary Q_f ; simultaneous actual substitution is certified	instantiated $(\lambda, u; \nu)$, or $\star$ when an exact actual is absent

A.5 Typing and size judgments

The typing judgment uses the primitive table and the following exhaustive constructors.

Shapes are **scalar**, products of shapes, or **array**($\lambda, u, N; \bar{\kappa}$) satisfying the certified constraints of section 4.3. The size judgment has the following exhaustive equations; J is recursive join and K_b is the body shape transformer checked under its index/accumulator binders.

A module check traverses declarations in order and stores only derivations of type, count/size, resource, representation, and memory admissibility. A missing derivation rejects the module; it is not a dynamic source assertion.

A.6 Evaluation, errors, and auxiliary relations

The rooted judgment is the one in section 5. Put $F_\rho(\vec{e}) = \text{rv}_\rho(\text{FV}(\vec{e}))$ and $\text{gc}_K(\text{ok}(v), \sigma) = \sigma \upharpoonright (K \cup \{v\})$, $\text{gc}_K(\text{err}(\epsilon), \sigma) = \sigma \upharpoonright K$. Ordered sequence evaluation carries every primary parameter:

$$\begin{array}{c}
 \frac{}{\Sigma; K \vdash \langle \langle \rangle, \rho, \sigma \rangle \Downarrow_{\text{seq}} \langle \text{ok}(\langle \rangle), \sigma \upharpoonright K \rangle \mid \mathbf{0}_1} \quad (\text{Seq0}) \\
 \\
 \frac{\begin{array}{c} \Sigma; K \cup F_\rho(\vec{e}) \vdash \langle e_1, \rho, \sigma \rangle \Downarrow \langle \text{ok}(v_1), \sigma_1 \rangle \mid C_1 \\ \Sigma; K \cup \{v_1\} \vdash \langle \vec{e}, \rho, \sigma_1 \rangle \Downarrow_{\text{seq}} \langle \text{ok}(\vec{v}), \sigma_2 \rangle \mid C_2 \end{array}}{\Sigma; K \vdash \langle e_1 :: \vec{e}, \rho, \sigma \rangle \Downarrow_{\text{seq}} \langle \text{ok}(v_1 :: \vec{v}), \sigma_2 \upharpoonright (K \cup \{v_1\} \cup \{\vec{v}\}) \rangle \mid C_1 \oplus C_2} \quad (\text{SeqS}) \\
 \\
 \frac{\Sigma; K \cup F_\rho(\vec{e}) \vdash \langle e_1, \rho, \sigma \rangle \Downarrow \langle \text{err}(\epsilon), \sigma_1 \rangle \mid C_1}{\Sigma; K \vdash \langle e_1 :: \vec{e}, \rho, \sigma \rangle \Downarrow_{\text{seq}} \langle \text{err}(\epsilon), \sigma_1 \upharpoonright K \rangle \mid C_1} \quad (\text{SeqErr1}) \\
 \\
 \frac{\begin{array}{c} \Sigma; K \cup F_\rho(\vec{e}) \vdash \langle e_1, \rho, \sigma \rangle \Downarrow \langle \text{ok}(v_1), \sigma_1 \rangle \mid C_1 \\ \Sigma; K \cup \{v_1\} \vdash \langle \vec{e}, \rho, \sigma_1 \rangle \Downarrow_{\text{seq}} \langle \text{err}(\epsilon), \sigma_2 \rangle \mid C_2 \end{array}}{\Sigma; K \vdash \langle e_1 :: \vec{e}, \rho, \sigma \rangle \Downarrow_{\text{seq}} \langle \text{err}(\epsilon), \sigma_2 \upharpoonright K \rangle \mid C_1 \oplus C_2} \quad (\text{SeqErrS})
 \end{array}$$

At the premise for child e_j , recursive K contains every earlier operand and F_ρ contains the values needed by the unexecuted suffix. Here $\{\vec{v}\}$ denotes the collection of values in the vector. No error constructor evaluates a later child. Peak p is derived from the entire enclosing trace under its original K , as fixed below; it is not recovered merely by taking the maximum of premise scalar peaks after earlier operands have been moved into a premise's continuation roots.

The terminal expression constructors are normative. Variables and constants are

$$\frac{\rho(x) = v}{\Sigma; K \vdash \langle x, \rho, \sigma \rangle \Downarrow \langle \text{ok}(v), \sigma \upharpoonright (K \cup \{v\}) \rangle \mid \mathbf{0}_1} \quad (\text{E-Var})$$

$$\frac{v = \text{bits}(c)}{\Sigma; K \vdash \langle c, \rho, \sigma \rangle \Downarrow \langle \text{ok}(v), \sigma \upharpoonright K \rangle \mid \mathbf{0}_1} \quad (\text{E-Const})$$

For a primitive, ordered operand evaluation is followed by exactly one primitive action:

$$\frac{\Sigma; K \vdash \langle \vec{e}, \rho, \sigma \rangle \Downarrow_{\text{seq}} \langle \text{ok}(\vec{v}), \sigma_1 \rangle \mid C_s \quad \delta_p(\vec{v}) = \text{ok}(v)}{\Sigma; K \vdash \langle p(\vec{e}), \rho, \sigma \rangle \Downarrow \langle \text{ok}(v), \text{gc}_K(\text{ok}(v), \sigma_1) \rangle \mid C_s \oplus \chi_p} \quad (\text{E-Prim})$$

$$\frac{\Sigma; K \vdash \langle \vec{e}, \rho, \sigma \rangle \Downarrow_{\text{seq}} \langle \text{ok}(\vec{v}), \sigma_1 \rangle \mid C_s \quad \delta_p(\vec{v}) = \text{err}(\epsilon)}{\Sigma; K \vdash \langle p(\vec{e}), \rho, \sigma \rangle \Downarrow \langle \text{err}(\epsilon), \text{gc}_K(\text{err}(\epsilon), \sigma_1) \rangle \mid C_s \oplus \chi_p} \quad (\text{E-PrimErr})$$

All forms with a displayed ordered child vector use the following constructor, instantiated with the rule names E-PrimArgErr, E-ProdArgErr, E-IndexArgErr, E-FoldArgErr, and E-CallArgErr as applicable:

$$\frac{\Sigma; K \vdash \langle \text{children}(F), \rho, \sigma \rangle \Downarrow_{\text{seq}} \langle \text{err}(\epsilon), \sigma_1 \rangle \mid C_s}{\Sigma; K \vdash \langle F, \rho, \sigma \rangle \Downarrow \langle \text{err}(\epsilon), \sigma_1 \upharpoonright K \rangle \mid C_s} \quad (\text{E-StrictErr}_F)$$

Table 7: Typing rules for every source expression form.

Rule	Premises	Conclusion
T-Var/T-Const	$\Gamma(x) = \tau$; literal has declared kind	$x : \tau$; $c : \text{type}(c)$
T-Prim	$\text{sig}(p) = \vec{\tau} \rightarrow \tau$, ordered operands have $\vec{\tau}$	$p(\vec{e}) : \tau$
T-Let	$e_1 : \tau_1$; under $x:\tau_1$, $e_2 : \tau_2$	let $x = e_1$ in $e_2 : \tau_2$
T-If	guard : bool ; both branches : τ	conditional : τ
T-Prod/Proj	$e_i : \tau_i$; product type and $1 \leq j \leq k$	tuple : $\prod_i \tau_i$; projection : τ_j
T-Array	$m \leq N$ and every $e_i : \tau$	literal : $\text{arr}\langle\tau, N\rangle$
T-Len/T-Index	array : $\text{arr}\langle\tau, N\rangle$; index : i32	length : i32 ; index result : τ
T-Fold	admissible count for N ; initial/body : τ_x under $i:\text{idx}(N)$, $x:\tau_x$	fold : τ_x
T-Build	admissible count for N ; body : τ under $i:\text{idx}(N)$	builder : $\text{arr}\langle\tau, N\rangle$
T-Call	earlier $f : \vec{\tau} \rightarrow \tau$; ordered actual types agree	$f(\vec{e}) : \tau$

Table 8: Result-size equations. “Scalar/count” means scalar shape plus the separate refinement in table 6.

Form	Premise or substitution	Result
scalar/variable	scalar literal/primitive; $\Gamma_{sz}(x) = \kappa$	scalar ; κ
product/proj.	component shapes κ_i ; valid field j	prod ($\vec{\kappa}$); κ_j
literal	$m \leq N$, element shapes κ_i	array (m, m, N ; $\bigsqcup_{i < m} \kappa_i$)
builder	count $(\lambda, u; \nu)$, body $\kappa_b(i)$	array (λ, u, N ; $\bigsqcup_{i < u} \kappa_b(i)$)
conditional	branch shapes κ_t, κ_f	$J(\kappa_t, \kappa_f)$; exact length retained iff equal
let	analyse body under $x \mapsto \kappa_1$	capture-avoiding body substitution
fold	$\kappa_0, \kappa_{i+1} = K_b(i, \kappa_i)$	κ_s for exact s ; otherwise $\bigsqcup_{0 \leq j \leq u} \kappa_j$
length/index	array array (λ, u, N ; $\bar{\kappa}$)	scalar/count (λ, u) ; $\bar{\kappa}$
call	earlier transformer K_f	simultaneous actual substitution, losing exactness to $\star$ as needed
capacity	type $\text{arr}\langle\tau, N\rangle$	array ($\star, N, N$; capshape (τ))
fallback		

Let and conditional rules declare future environment roots before their first premise. Put $K_2 = K \cup \text{rv}_\rho(\text{FV}(e_2) \setminus \{x\})$ and $K_{tf} = K \cup \text{rv}_\rho(\text{FV}(e_t) \cup \text{FV}(e_f))$. Then

$$\frac{\Sigma; K_2 \vdash \langle e_1, \rho, \sigma \rangle \Downarrow \langle \text{ok}(v_1), \sigma_1 \rangle \mid C_1 \quad \Sigma; K \vdash \langle e_2, \rho[x \mapsto v_1], \sigma_1 \rangle \Downarrow \langle r, \sigma_2 \rangle \mid C_2}{\Sigma; K \vdash \langle \text{let } x = e_1 \text{ in } e_2, \rho, \sigma \rangle \Downarrow \langle r, \text{gc}_K(r, \sigma_2) \rangle \mid C_1 \oplus C_2} \quad (\text{E-Let})$$

$$\frac{\Sigma; K_2 \vdash \langle e_1, \rho, \sigma \rangle \Downarrow \langle \text{err}(\epsilon), \sigma_1 \rangle \mid C_1}{\Sigma; K \vdash \langle \text{let } x = e_1 \text{ in } e_2, \rho, \sigma \rangle \Downarrow \langle \text{err}(\epsilon), \sigma_1 \upharpoonright K \rangle \mid C_1} \quad (\text{E-LetErr})$$

$$\frac{\Sigma; K_{tf} \vdash \langle e_c, \rho, \sigma \rangle \Downarrow \langle \text{ok}(b), \sigma_1 \rangle \mid C_c \quad b = \text{true} \quad \Sigma; K \vdash \langle e_t, \rho, \sigma_1 \rangle \Downarrow \langle r, \sigma_2 \rangle \mid C_t}{\Sigma; K \vdash \langle \text{if } e_c \text{ then } e_t \text{ else } e_f, \rho, \sigma \rangle \Downarrow \langle r, \text{gc}_K(r, \sigma_2) \rangle \mid C_c \oplus \chi_{\text{decision}} \oplus C_t} \quad (\text{E-IfT})$$

Rule E-IfF is identical with $b = \text{false}$ and premise e_f ; E-IfErr returns a guard error restricted to K and adds no decision action. Explicitly, these two constructors are

$$\frac{\Sigma; K_{tf} \vdash \langle e_c, \rho, \sigma \rangle \Downarrow \langle \text{ok}(\text{false}), \sigma_1 \rangle \mid C_c \quad \Sigma; K \vdash \langle e_f, \rho, \sigma_1 \rangle \Downarrow \langle r, \sigma_2 \rangle \mid C_f}{\Sigma; K \vdash \langle \text{if } e_c \text{ then } e_t \text{ else } e_f, \rho, \sigma \rangle \Downarrow \langle r, \text{gc}_K(r, \sigma_2) \rangle \mid C_c \oplus \chi_{\text{decision}} \oplus C_f} \quad (\text{E-IfF})$$

$$\frac{\Sigma; K_{tf} \vdash \langle e_c, \rho, \sigma \rangle \Downarrow \langle \text{err}(\epsilon), \sigma_1 \rangle \mid C_c}{\Sigma; K \vdash \langle \text{if } e_c \text{ then } e_t \text{ else } e_f, \rho, \sigma \rangle \Downarrow \langle \text{err}(\epsilon), \sigma_1 \upharpoonright K \rangle \mid C_c} \quad (\text{E-IfErr})$$

Products, projection, length, and indexing are fixed by

$$\frac{\Sigma; K \vdash \langle \vec{e}, \rho, \sigma \rangle \Downarrow_{\text{seq}} \langle \text{ok}(\vec{v}), \sigma_1 \rangle \mid C_s}{\Sigma; K \vdash \langle (\vec{e}), \rho, \sigma \rangle \Downarrow \langle \text{ok}((\vec{v})), \text{gc}_K(\text{ok}((\vec{v})), \sigma_1) \rangle \mid C_s \oplus \chi_{\text{prod}}} \quad (\text{E-Prod})$$

$$\frac{\Sigma; K \vdash \langle e, \rho, \sigma \rangle \Downarrow \langle \text{ok}((v_1, \dots, v_k)), \sigma_1 \rangle \mid C \quad 1 \leq j \leq k}{\Sigma; K \vdash \langle \pi_j(e), \rho, \sigma \rangle \Downarrow \langle \text{ok}(v_j), \text{gc}_K(\text{ok}(v_j), \sigma_1) \rangle \mid C \oplus \chi_{\text{proj}}} \quad (\text{E-Proj})$$

$$\frac{\Sigma; K \vdash \langle e, \rho, \sigma \rangle \Downarrow \langle \text{ok}(\alpha), \sigma_1 \rangle \mid C \quad \sigma_1(\alpha) = \text{seal}(\tau, N, \ell, \omega, \vec{v})}{\Sigma; K \vdash \langle \text{len}(e), \rho, \sigma \rangle \Downarrow \langle \text{ok}(\text{i32}(\ell)), \sigma_1 \upharpoonright K \rangle \mid C \oplus \chi_{\text{len}}} \quad (\text{E-Len})$$

$$\frac{\Sigma; K \vdash \langle e_a, e_i, \rho, \sigma \rangle \Downarrow_{\text{seq}} \langle \text{ok}(\alpha, j), \sigma_1 \rangle \mid C_s \quad \sigma_1(\alpha) = \text{seal}(\tau, N, \ell, \omega, \vec{v}) \quad 0 \leq j < \ell}{\Sigma; K \vdash \langle e_a[e_i], \rho, \sigma \rangle \Downarrow \langle \text{ok}(v_j), \text{gc}_K(\text{ok}(v_j), \sigma_1) \rangle \mid C_s \oplus \chi_{\text{indexOk}, \tau}} \quad (\text{E-Index})$$

$$\frac{\Sigma; K \vdash \langle e_a, e_i, \rho, \sigma \rangle \Downarrow_{\text{seq}} \langle \text{ok}(\alpha, j), \sigma_1 \rangle \mid C_s \quad \sigma_1(\alpha) = \text{seal}(\tau, N, \ell, \omega, \vec{v}) \quad (j < 0 \vee j \geq \ell)}{\Sigma; K \vdash \langle e_a[e_i], \rho, \sigma \rangle \Downarrow \langle \text{err}(\text{Bounds}), \sigma_1 \upharpoonright K \rangle \mid C_s \oplus \chi_{\text{indexErr}}} \quad (\text{E-IndexErr})$$

Child errors for projection and length use the corresponding direct one-premise propagation instance of E-StrictErr. The table below is an index to these normative constructors and to the literal/fold/builder/call constructors that follow; it is not a substitute for them.

For $R_N = \text{region}(\tau, N)$, literal entry and its distinct-expression prefix relation are

$$\begin{array}{c}
\frac{\alpha \notin \text{dom}(\sigma) \quad m \leq N \quad \sigma_0 = \sigma[\alpha \mapsto \text{pref}(\tau, N, m, 0, \text{local}, \langle \rangle)]}{\Sigma; K \vdash \langle \vec{e}, \rho, \sigma_0, 0, m, \alpha \rangle \Downarrow_{\text{lit}} \langle r, \sigma_1 \rangle \mid C_l} \\
\hline
\Sigma; K \vdash \langle [\vec{e}]_N, \rho, \sigma \rangle \Downarrow \langle r, \text{gc}_K(r, \sigma_1) \rangle \mid \chi_{\text{reserve}}(R_N) \oplus C_l \quad (\text{E-Lit})
\end{array}$$

$$\begin{array}{c}
j = m \quad \sigma(\alpha) = \text{pref}(\tau, N, m, m, \omega, \vec{v}) \\
\hline
\Sigma; K \vdash \langle \vec{e}, \rho, \sigma, j, m, \alpha \rangle \Downarrow_{\text{lit}} \langle \text{ok}(\alpha), \sigma[\alpha \mapsto \text{seal}(\tau, N, m, \omega, \vec{v})] \uparrow (K \cup \{\alpha\}) \rangle \mid \chi_{\text{seal}} \quad (\text{Lit0})
\end{array}$$

$$\begin{array}{c}
j < m \quad K_j = K \cup \{\alpha\} \cup F_\rho(e_{j+1}, \dots, e_{m-1}) \\
\Sigma; K_j \vdash \langle e_j, \rho, \sigma \rangle \Downarrow \langle \text{ok}(v_j), \sigma_1 \rangle \mid C_j \\
\sigma_2 = \sigma_1[\alpha \mapsto \text{pref}(\tau, N, m, j+1, \omega, \vec{v}_{v_j})] \\
\Sigma; K \vdash \langle \vec{e}, \rho, \sigma_2, j+1, m, \alpha \rangle \Downarrow_{\text{lit}} \langle r, \sigma_3 \rangle \mid C_r \\
\hline
\Sigma; K \vdash \langle \vec{e}, \rho, \sigma, j, m, \alpha \rangle \Downarrow_{\text{lit}} \langle r, \sigma_3 \rangle \mid C_j \oplus \chi_{\text{elementWrite}, \tau} \oplus C_r \quad (\text{LitS})
\end{array}$$

$$\begin{array}{c}
j < m \quad K_j = K \cup \{\alpha\} \cup F_\rho(e_{j+1}, \dots, e_{m-1}) \\
\Sigma; K_j \vdash \langle e_j, \rho, \sigma \rangle \Downarrow \langle \text{err}(\epsilon), \sigma_1 \rangle \mid C_j \\
\hline
\Sigma; K \vdash \langle \vec{e}, \rho, \sigma, j, m, \alpha \rangle \Downarrow_{\text{lit}} \langle \text{err}(\epsilon), \sigma_1 \uparrow K \rangle \mid C_j \quad (\text{LitErr})
\end{array}$$

Thus a literal reserves once, performs one write after each successful distinct element, and seals once after m successes. LitErr performs neither the failing-position write nor a seal, and its restriction removes the unreachable prefix object. No rule contains an index test or repeats a common body.

Fold entry is the rule

$$\begin{array}{c}
K_b = K \cup \text{rv}_\rho(\text{FV}(e_b) \setminus \{i, x\}) \\
\Sigma; K_b \vdash \langle e_n, e_0, \rho, \sigma \rangle \Downarrow_{\text{seq}} \langle \text{ok}(\text{i32}(n), v_0), \sigma_1 \rangle \mid C_s \\
0 \leq n \leq N \quad \Sigma; K_b \vdash \langle e_b, \rho, \sigma_1, i, x, 0, n, v_0 \rangle \Downarrow_{\text{fold}} \langle r, \sigma_2 \rangle \mid C_f \\
\hline
\Sigma; K \vdash \langle \text{fold}_N(e_n; i, x; e_0; e_b), \rho, \sigma \rangle \Downarrow \langle r, \text{gc}_K(r, \sigma_2) \rangle \mid C_s \oplus C_f \quad (\text{E-Fold})
\end{array}$$

The fold base, success, and failure rules are E-Fold0/E-FoldS/E-FoldErr in section 5. Builder judgments are parameterised by ambient roots K and have the following rules:

$$\begin{array}{c}
j = n \quad \sigma(\alpha) = \text{pref}(\tau, N, n, n, \omega, \vec{v}) \\
\hline
\Sigma; K \vdash \langle e_b, \rho, \sigma, i, j, n, \alpha \rangle \Downarrow_{\text{build}} \\
\langle \text{ok}(\alpha), \sigma[\alpha \mapsto \text{seal}(\tau, N, n, \omega, \vec{v})] \uparrow (K \cup \{\alpha\}) \rangle \mid \chi_{\text{loopTest}} \oplus \chi_{\text{seal}} \quad \text{BUILD0},
\end{array}$$

$$\begin{array}{c}
j < n \quad \Sigma; K \cup \{\alpha\} \vdash \langle e_b, \rho[i \mapsto j], \sigma \rangle \Downarrow \langle \text{ok}(v_j), \sigma_1 \rangle \mid C_b \\
\sigma_2 = \sigma_1[\alpha \mapsto \text{pref}(\tau, N, n, j+1, \omega, \vec{v}_{v_j})] \\
\Sigma; K \vdash \langle e_b, \rho, \sigma_2, i, j+1, n, \alpha \rangle \Downarrow_{\text{build}} \langle r, \sigma' \rangle \mid C_r \\
\hline
\Sigma; K \vdash \langle e_b, \rho, \sigma, i, j, n, \alpha \rangle \Downarrow_{\text{build}} \langle r, \sigma' \rangle \mid \chi_{\text{loopTest}} \oplus C_b \oplus \chi_{\text{elementWrite}, \tau} \oplus C_r \quad \text{BUILDS}.
\end{array}$$

The failing builder rule is

$$\begin{array}{c}
j < n \quad \Sigma; K \cup \{\alpha\} \vdash \langle e_b, \rho[i \mapsto j], \sigma \rangle \Downarrow \langle \text{err}(\epsilon), \sigma_1 \rangle \mid C_b \\
\hline
\Sigma; K \vdash \langle e_b, \rho, \sigma, i, j, n, \alpha \rangle \Downarrow_{\text{build}} \langle \text{err}(\epsilon), \sigma_1 \uparrow K \rangle \mid \chi_{\text{loopTest}} \oplus C_b \quad \text{BUILDErr}.
\end{array}$$

The partial prefix is unreachable from K and is removed at the boundary. Builder entry is

$$\begin{array}{c}
K_b = K \cup \text{rv}_\rho(\text{FV}(e_b) \setminus \{i\}) \quad \Sigma; K_b \vdash \langle e_n, \rho, \sigma \rangle \Downarrow \langle \text{ok}(\text{i32}(n)), \sigma_1 \rangle \mid C_n \\
0 \leq n \leq N \quad \alpha \notin \text{dom}(\sigma_1) \\
\sigma_0 = \sigma_1[\alpha \mapsto \text{pref}(\tau, N, n, 0, \text{local}, \langle \rangle)] \\
\Sigma; K_b \vdash \langle e_b, \rho, \sigma_0, i, 0, n, \alpha \rangle \Downarrow_{\text{build}} \langle r, \sigma_2 \rangle \mid C_b \\
\hline
\Sigma; K \vdash \langle \text{build}_N(e_n; i.e_b), \rho, \sigma \rangle \Downarrow \langle r, \text{gc}_K(r, \sigma_2) \rangle \mid C_n \oplus \chi_{\text{reserve}}(R_N) \oplus C_b \quad (\text{E-Build})
\end{array}$$

E-BuildCountErr is the direct propagation rule for an error from e_n under K_b , restricted to the original K , with no reservation. The builder auxiliary alone repeats e_b .

Finally, if $\Sigma(f) = (\vec{x}:\vec{\tau}, e_f)$ and $f \prec_M g$, direct-call success and callee-error propagation are

$$\frac{\begin{array}{c} \Sigma; K \vdash \langle \vec{e}, \rho, \sigma \rangle \Downarrow_{\text{seq}} \langle \text{ok}(\vec{v}), \sigma_1 \rangle \mid C_s \\ \Sigma; K \cup \{\vec{v}\} \vdash \langle e_f, [\vec{x} \mapsto \vec{v}], \sigma_1 \rangle \Downarrow \langle \text{ok}(v), \sigma_2 \rangle \mid C_f \end{array}}{\Sigma; K \vdash \langle f(\vec{e}), \rho, \sigma \rangle \Downarrow \langle \text{ok}(v), \text{gc}_K(\text{ok}(v), \sigma_2) \rangle \mid C_s \oplus \text{liftCall}(C_f)} \quad (\text{E-Call})$$

$$\frac{\begin{array}{c} \Sigma; K \vdash \langle \vec{e}, \rho, \sigma \rangle \Downarrow_{\text{seq}} \langle \text{ok}(\vec{v}), \sigma_1 \rangle \mid C_s \\ \Sigma; K \cup \{\vec{v}\} \vdash \langle e_f, [\vec{x} \mapsto \vec{v}], \sigma_1 \rangle \Downarrow \langle \text{err}(\epsilon), \sigma_2 \rangle \mid C_f \end{array}}{\Sigma; K \vdash \langle f(\vec{e}), \rho, \sigma \rangle \Downarrow \langle \text{err}(\epsilon), \sigma_2 \upharpoonright K \rangle \mid C_s \oplus \text{liftCall}(C_f)} \quad (\text{E-CallErr})$$

E-CallArgErr is E-StrictErr for the actual vector and adds no call event. Function `liftCall` adds $\chi_{\text{callReturn}}$ and increases relative active depth by one, as defined next.

A.7 Dynamic events and static resource equations

Let $H = 8$, let $w(\text{bool}) = w(\text{i32}) = 4$ and $w(\text{i64}) = w(\text{f64}) = 8$. Product width is the aligned field sum; an aggregate field is a 4-byte offset. Define $\text{region}(\tau, N) = \text{align}(H + Nw(\tau), A(\tau))$.

Let $\mathbf{0}_1 = \langle 0, 0, 0, 0, 0, 0, 0, 1 \rangle$. Every expression and auxiliary derivation occurs in one current source activation, so its depth component is at least one even when it contains no terminal action. Terminal rows affect the seven non-depth components displayed in table 10. If C_f is a callee-body cost relative to its own activation, `liftCall`(C_f) adds $\chi_{\text{callReturn}}$ and replaces d_f by $d_f + 1$. An exported-entry invocation evaluates its body in the initial activation ($d = 1$) but belongs to the ABI boundary and adds no $\chi_{\text{callReturn}}$. Only an explicit source call expression uses `liftCall`.

For a derivation, cumulative components sum its terminal rows, depth follows the rule above, and peak is the maximum rooted local-region sum over all premise boundaries and literal/fold/builder administrative states under the enclosing rule's original K . In particular, a partial literal and its completed element graphs remain live while the next distinct expression runs. This trace definition fixes dynamic C independently of an implementation; peak is not reconstructed from scalar subcosts after their retained roots have been reclassified as continuation roots.

The static judgment returns $\Phi = (E, L, P)$ as in section 6. Let $\text{seq}(\Phi_1, \Phi_2) = (E_1 \oplus_E E_2, L_2, \max(P_1, P_2))$ where Φ_2 is checked under all footprints retained from the first result; let branch use componentwise maximum. The exhaustive equations are:

$$\begin{aligned} \Phi(x) &= (\mathbf{0}_1^E, \Lambda(x), \Lambda(x)), & \Phi(c) &= (\mathbf{0}_1^E, 0, 0), \\ \Phi(p(\vec{e})) &= \text{SeqA}(\vec{e}) \oplus \chi_p, & \Phi((\vec{e})) &= \text{SeqA}(\vec{e}) \oplus \chi_{\text{prod}}, \\ \Phi(\pi_j(e)) &= \Phi(e) \oplus \chi_{\text{proj}}, & \Phi(\text{len}(e)) &= \Phi(e) \oplus \chi_{\text{len}}, \\ \Phi(e_a[e_i]) &= \text{SeqA}(e_a, e_i) \oplus (\chi_{\text{indexOk}} \sqcup \chi_{\text{indexErr}}), \\ \Phi(\text{let } x = e_1 \text{ in } e_2) &= \text{seq}(\Phi_1, \Phi_2[\Lambda, x \mapsto L_1]), \\ \Phi(\text{if } e_c \text{ then } e_t \text{ else } e_f) &= \Phi_c \oplus \chi_{\text{decision}} \oplus (\Phi_t \sqcup \Phi_f), \\ \Phi(f(\vec{e})) &= \text{SeqA}(\vec{e}) \oplus \text{liftCall}(\Phi_f[\vec{n}, \vec{L} := \vec{u}, \vec{L}_e]). \end{aligned}$$

$\mathbf{0}_1^E$ has zero cumulative components and depth one. `SeqA` is the finite left-to-right fold of `seq` and includes the footprints of already evaluated operands in every later P .

Literal has a distinct equation. For element summaries $\Phi_j = (E_j, L_j, P_j)$, put $A_0 = R_N$ and $A_{j+1} = A_j + L_j$. Analyse e_j with the partial literal mapped to A_j and with its suffix free-variable

Table 9: Index of expression-specific normative evaluation constructors.

Form	Evaluated premises	Unique terminal result/action
x, c	none	$\text{ok}(\rho(x))$ or $\text{ok}(\text{bits}(c))$
$p(\vec{e})$	$\text{evalSeq}(\vec{e})$	$\delta_p(\vec{v})$, either value or declared arithmetic error
let	e_1 , then e_2 under $x \mapsto v_1$	body result; v_1 retained while required
if	guard, then exactly the branch named by its Boolean	selected branch result
product/proj.	all fields; one product operand	tuple; field j
array literal	one fresh prefix, then the distinct fields in order	literal-prefix auxiliary; seal at prefix m and return α
length	one sealed array α	nonnegative i32 word for ℓ
index	sealed α , then signed index j	$\text{ok}(v_j)$ if $0 \leq j < \ell$; otherwise $\text{err}(\text{Bounds})$
fold	count, then initial accumulator	auxiliary $\Downarrow_{\text{fold}}$ at $(k, n, v) = (0, n, v_0)$
builder	count, then fresh prefix α	auxiliary $\Downarrow_{\text{build}}$ at $(j, n, \alpha) = (0, n, \alpha)$
call	actuals in order	body of the uniquely named earlier declaration under formal-to-actual environment

Table 10: Terminal dynamic event components. Active depth is supplied structurally rather than by these rows.

Event	o	a	h	r_b	w_b	q
primitive; product/proj.	1	0	0	0	0	0
conditional or loop test	1	0	0	0	0	1
reserve region R	1	1	R	0	0	0
write element τ	1	0	0	0	$w(\tau)$	0
seal header	1	0	0	0	H	0
length	1	0	0	4	0	0
successful index τ	1	0	0	$4 + w(\tau)$	0	1
failed index	1	0	0	4	0	1
call and return	2	0	0	0	0	0
variable, constant, propagation	0	0	0	0	0	0

footprints retained. Then

$$\begin{aligned}
L_{\text{lit}} &= A_m, \\
P_{\text{lit}} &= \max \left(R_N, A_m, \max_{0 \leq j < m} P_j[\Lambda(\alpha) \mapsto A_j] \right), \\
E_{\text{lit}}^{\text{ok}} &= \chi_{\text{reserve}}(R_N) \oplus_E \bigoplus_{j=0}^{m-1} (E_j \oplus_E \chi_{\text{elementWrite}, \tau}) \oplus_E \chi_{\text{seal}}, \\
E_{\text{lit}, t}^{\text{err}} &= \chi_{\text{reserve}}(R_N) \oplus_E \bigoplus_{j=0}^{t-1} (E_j \oplus_E \chi_{\text{elementWrite}, \tau}) \oplus_E E_t, \quad 0 \leq t < m. \quad (\text{C-Lit})
\end{aligned}$$

The candidate event bound joins the successful vector with every finite $E_{\text{lit}, t}^{\text{err}}$; its peak likewise takes the corresponding prefix maximum. No χ_{loopTest} occurs. Fold and build are exactly C-Fold-Peak and C-Build of section 6, including count/initial costs, loop tests, reservation, writes, seal, result footprints, and capacity-bounded maxima/sums. Error-prefix bounds are the maximum of every finite ordered prefix and the successful terminal case; nonnegative cumulative rows permit a full-prefix upper expression, but peak still uses the explicit prefix maximum. These equations reject a summary whose retained-root transformer is unavailable.

A.8 Layout admissibility

All layout arithmetic is performed in $\mathbb{N}$. For power-of-two A , $\text{align}(x, A) = A \lceil x/A \rceil$. The predicates are

$$\begin{aligned}
\text{CapOK}(N) &\equiv N \leq 2^{31} - 1, \\
\text{RegionOK}(b, z) &\equiv b < 2^{32} \wedge b + z \leq 2^{32}, \\
\text{Disjoint}(b, z, b', z') &\equiv b + z \leq b' \vee b' + z' \leq b.
\end{aligned}$$

ABI validation checks scalar/header encodings, declared capacities and logical lengths, initialised logical bytes, alignment, every endpoint, and required separation. A failed check yields `InvalidABI` before the source entry. Products, alignments, page rounding, increments, and $Nw(\tau)$ require checked natural-number derivations before any 32-bit word is formed.

B BIR Machine, Liveness Certificates, and Lowering

This appendix fixes the mathematical objects quantified over by the translation proof obligations. It is not an executable IR or implementation algorithm.

B.1 BIR syntax and configurations

BIR types, capabilities, operands, instructions, and terminators are

$$\begin{aligned}
\theta &::= \text{bool} \mid \text{i32} \mid \text{i64} \mid \text{f64} \mid \text{prod}(\vec{\theta}) \mid \text{ptr}(\tau, N, c), \\
c &::= \text{inR}(b, z) \mid \text{fresh}(\alpha, b, z, n, j) \mid \text{imm}(\alpha, b, z, \ell) \mid \text{scratch}(b, z) \mid \text{outW}(b, z), \\
a &::= r \mid k, \\
I &::= r := \text{const } k \mid r := \text{prod}(\vec{a}) \mid r := \text{proj}_j(a) \\
&\quad \mid r := p(\vec{a}) \triangleright \vec{L}_\epsilon \mid r := \text{addr}(c, a, z) \mid r := \text{load}_\theta(c, a) \mid r := \text{reserve}(c, N, n) \\
&\quad \mid \text{init}_\theta(c, a, a) \mid c' := \text{seal}(c) \mid \text{storeOut}_\theta(c, a, a), \\
T &::= \text{br } L(\vec{a}) \mid \text{cbr } a \ L_t(\vec{a}) \ L_f(\vec{a}) \mid \text{fail } \epsilon \mid \text{return } a \mid \text{call } f(\vec{a}; c_o) \text{ to } L_s(r) \text{ else } \vec{L}_\epsilon.
\end{aligned}$$

Here b, z, n, j in a capability are mathematical metadata checked against the corresponding runtime words; programs cannot inspect a capability. A block is $L(\vec{r}; \vec{\theta}) = I_0 \cdots I_{h-1}; T$; a function contains a finite entry-labelled block map, typed input/output descriptors, a frame layout, and an earlier-declaration rank. A BIR module is a finite function map with one entry and an injective declared-error status map.

A frame is $\varphi = \langle f, L, k, \nu, b_F, \mathcal{C}, c_o, L_s, \vec{L}_\epsilon \rangle$: k is the next instruction, ν an SSA valuation, b_F a frame base, $\mathcal{C}$ the live capability map, and the final fields the caller output/continuations. A configuration is

$$S_I = \langle \Pi, \varphi, m \rangle \quad \text{or} \quad \text{final}_I(s, m) \quad \text{or} \quad \text{fault}_I(\kappa),$$

where Π is a finite caller-frame stack and

$$m : [0, 2^{32}) \rightarrow \{0, 1\}^8.$$

Fault classes are

$$\kappa \in \{\text{badType}, \text{badCap}, \text{uninit}, \text{oob}, \text{badControl}, \text{stackExhaust}\}.$$

The initial configuration has one entry frame, empty SSA valuation except decoded parameters, checked input/output capabilities, the validated memory image, and stack ϵ . A final success is status zero plus completed output; a declared error is its nonzero injective status with output ignored.

Write $\nu(a)$ for operand lookup, $\text{bytes}_\theta(v)$ for the fixed representation, and $\text{inside}(c, o, z)$ when $[o, o + z)$ lies inside the interval named by c . Failed dynamic premises are classified by the total map **faultOf** below. The checks are applied in the displayed order; hence a state violating several premises still has one fault outcome.

Operand/function arity is a type failure; labelled-block arity is a control failure. The small-step relation is the least relation generated by table 12 together with this ordered failure map.

A trap-prone scalar division is not a BIR primitive step until its zero/overflow guards have branched to declared errors. No successful step reads an unsealed builder, writes an immutable/input capability, grows memory, calls indirectly, or accesses an import. Validation is intended to rule out **badControl**, **badType**, **badCap**, **uninit**, and **oob** for invariant-related reachable states, while configured activation capacity is intended to exclude **stackExhaust**. These are premises of **PO-NOFAULT**, not consequences proved by defining the map.

B.2 BIR validation

Judgment $\Xi; \Gamma_L; \Gamma_r; \mathcal{C}; q \vdash I \Rightarrow (\Gamma'_r, \mathcal{C}')$ checks one instruction, and $\Xi; \Gamma_L; \Gamma_r; \mathcal{C} \vdash T \text{ ok}$ checks a terminator. Its clauses are the static counterparts of every premise in table 12: unique SSA

Table 11: Total BIR premise-to-fault map. A primitive’s declared arithmetic error takes its declared error edge and is not a BIR fault.

Check order and class	Failed premise	Result
1. control fetch	current frame, block, instruction position, label, continuation depth, block arity, direct target, rank decrease, or control-state shape is invalid	badControl
2. type	absent SSA operand, literal/destination type, primitive/function signature or function arity, product/projection field type, branch-argument type, condition type, load/store width, or return type disagrees	badType
3. capability	capability is absent, has the wrong permission/kind, aliases a simultaneously live capability at reservation, or attempts an input/immutable write	badCap
4. initialisation	load addresses a logically uninitialised fresh element; init does not target the unique next prefix; seal occurs before $j = n$; return/output reads an incomplete result	uninit
5. interval	natural offset, width, count-derived region, checked sum, alignment, or endpoint lies outside the named capability or $[0, 2^{32})$	oob
6. activation space	a statically required callee activation cannot be placed within the configured frame/linear-memory limit	stackExhaust

Table 12: BIR transition rules. Advancing changes k to $k + 1$; entering a block binds its arguments simultaneously and sets $k = 0$.

Form	Dynamic premises	Successor action
const	destination has literal type	bind literal; advance
primitive	operands have signature; $\delta_p(\vec{v})$ succeeds	bind result; advance; a declared arithmetic error enters its error block
product/proj.	field types/arity agree	construct/select value; advance
address	natural offset and width are inside capability; checked sum is below 2^{32}	bind address $b + o$ with inherited interval; advance
load	capability is <code>inR</code> , <code>imm</code> , or initialised portion of <code>fresh</code> ; bytes inside	decode bytes; advance
reserve	capability interval is frame scratch or output, $0 \leq n \leq N$, region is disjoint from every live capability	replace by <code>fresh</code> ($\alpha, b, z, n, 0$); advance
init	capability is <code>fresh</code> (α, b, z, n, j), $j < n$, address is element j	write one representation; change prefix to $j + 1$; advance
seal	capability is <code>fresh</code> (α, b, z, n, n)	write length/capacity header; replace by <code>imm</code> (α, b, z, n); advance
storeOut	output capability, inside address, fully computed value	write result bytes; advance
branch	target arity/types agree	enter target with simultaneous arguments
cbr	Boolean operand	enter exactly the true or false target
fail	ϵ has a declared status	unwind frames without reading output; finalise status at entry
call	rank decreases; actual/callee types agree; callee frame endpoint fits; output capability agrees	push caller continuation; enter callee with fresh frame base and actuals
return	result has declared type and output is complete	if caller exists, pop/bind result and enter success continuation; otherwise finalise zero status

definition and dominance, exact operand type, target arity/type, capability interval and permission, initialised-before-read, unique-prefix write, seal only at $j = n$, output completeness, decreasing call rank, and checked frame endpoints. Block validation folds the instruction judgment and then checks its terminator; function validation checks all reachable blocks, unique entry, one terminator, and loop certificates. Module validation checks the finite function map, status injection, memory layout, and one export.

A loop certificate names header L_h , latch L_l , exit L_x , capacity N , count register n , and induction register k . Validation requires entry $k = 0$, invariant $0 \leq k \leq n \leq N$, the only back edge $k' = k + 1$ under $k < n$, and exit only under $k = n$. A builder certificate additionally requires prefix $j = k$, one init on every successful body path, no prefix write on an error path, and one seal on exit.

Because validation judgments inspect finite maps and finite certificate trees, checking terminates. This fact does not show that source lowering produces a valid BIR module; that is PO-IR-WF.

B.3 Liveness and slot-colouring certificates

Normalised points form a finite set Q . Dataflow roots are defined backwards by

$$\text{Roots}(q) = \text{Use}(q) \cup \bigcup_{q' \in \text{succ}(q)} (\text{Roots}(q') \setminus \text{Def}(q)),$$

with function results, error continuations, actuals, literal/builder prefixes, and accumulators included at their corresponding points. The certificate supplies a finite solution R_q and, for every equation, two checked inclusions proving equality. Object α is live at q exactly when $\alpha \in \text{Reach}(R_q)$ in the normalised source shape graph.

A placement certificate $\mathcal{L}$ assigns each reservation α a frame offset, extent, and alignment $(o_\alpha, z_\alpha, A_\alpha)$. Judgment

$$F \vdash \mathcal{L} : \text{colour}(Q, R)$$

holds iff every offset is aligned, $o_\alpha + z_\alpha \leq F$, z_α dominates the capacity region, and every pair simultaneously live at any q has disjoint half-open intervals. A reuse edge from dead α to fresh β additionally carries a checked point q with $\alpha \notin \text{Reach}(R_q)$ and with no live capability to its interval. The source–target injection removes α before adding β . The fallback assigns reservations in source order to successively aligned, nonoverlapping intervals; pairwise disjointness then holds independently of liveness, at the cost of a larger F .

B.4 Source-to-BIR graph constructors

A fragment is $G = \langle B, L_{in}, L_{ok}(r), \vec{L}_\epsilon \rangle$. Graph operations are typed sequential composition $G_1;_r G_2$, disjoint union with fresh labels $G_1 \uplus G_2$, conditional join $\text{If}G$, certified counted loop $\text{Loop}G_N$, and certified builder loop $\text{Build}G_N$. These are finite graph constructors: composition redirects only L_{ok} ; error labels are shared and never redirected through a later source action.

The inductive judgment

$$\Sigma; \Delta; \Gamma \vdash e \Downarrow_{SI} \langle G, L_{in}, L_{ok}(r), \vec{L}_\epsilon \rangle$$

has one constructor per source form:

Formally, $\text{Lit}G_N(G_0, \dots, G_{m-1})$ creates one reserve node and a finite acyclic chain. Chain position j enters the distinct fragment G_j ; its success continuation performs exactly one init at element j and enters position $j + 1$, while every error continuation bypasses all later fragments and seal. Position m performs exactly one seal and exposes the pointer. It contains no conditional or

Table 13: Source-to-BIR constructors. **SeqG** is left-to-right typed composition with all error labels shared.

Source form	Graph constructor	Required certificate/invariant
c, x	one const node; existing SSA value	literal/variable type
$p(\vec{e})$	SeqG ($\vec{G}$); $r := p(\vec{r})$ with division guards where required	primitive signature, canonical-source-NaN relation
let	$G_1; r_1 \ G_2[x \mapsto r_1]$	capture-free SSA substitution
if	$G_c; \text{cbr } r_c \ L_t \ L_f$ plus $G_t \uplus G_f$ and typed join	one selected successor, common result type
product/proj. literal	SeqG ($\vec{G}$) plus product/projection node LitG_N ($G_0, \dots, G_{m-1}$)	fixed field representation one reservation; each distinct fragment once; one write per success; no loop test
length index	G ; load_{i32} at header length offset SeqG (G_a, G_i); length load; signed nonnegative/upper checks; checked address; load	immutable/input capability bounds edge precedes address; static certificate may discharge checks
fold	SeqG (G_n, G_0); LoopG_N (G_b) with header (k, n, x)	checked count and loop certificate
builder	G_n ; reserve ; BuildG_N (G_b); seal	count, prefix, capability, and layout certificates
call	SeqG ($\vec{G}$); call $f(\vec{r}; c_o)$	decreasing rank; caller-owned aggregate destination
result/status	designated typed continuation; declared-error continuation	output complete only on success
death/reuse	injection-removal annotation before a new reserve	accepted liveness certificate, or no-reuse placement

loop certificate. In contrast, $\text{LoopG}_N(G_b)$ creates fresh preheader, header, body, latch, and exit labels; the header block arguments are exactly (k, n, x) , its branch is $k < n$, the latch argument is $(k + 1, n, x')$, and the exit requires $k = n$. BuildG_N uses (j, n, c) , repeats the single G_b , inserts exactly one init between successful body and latch, sends every error directly to its shared label, and makes the sealed pointer available only at exit. These equations, rather than a compiler implementation, define the constructors.

B.5 BIR-to-Wasm constructors

Let $\mathcal{W}[\![\!-\!]\!]_\lambda$ map a validated BIR function and a structuring certificate λ to a sequence in the supported scalar Wasm subset. The value map places scalar SSA values in typed locals or operand-stack positions and pointers in i32 locals. The state map places input, output, frames, and runtime reserve in the intervals of (2). Instruction constructors are:

Certificate λ is a finite tree of block, loop, conditional, and branch nodes. Its checker proves that each reachable BIR block appears once in the structured region (except a certified loop header reached by its unique back edge), each BIR edge maps to the indicated continuation depth, and administrative branches carry no source/BIR action. The induced relation $G \Downarrow_{IW}^\lambda W$ holds exactly when module validation, constructor traversal, memory placement, and λ checking all succeed. It admits no imports, memory growth, table, indirect call, reference type, shared memory, thread, atomic operation, exception, or relaxed SIMD instruction.

The above definitions make PO-SI-EXPR, PO-IW, and PO-NOFAULT well formed. They do not discharge those obligations: in particular, the exhaustive simulation of every constructor, aggregate alias case, and structured-control path remains future proof work.

Table 14: BIR-to-Wasm mathematical constructors for the supported subset.

BIR form	Wasm constructor	Side condition
const/local/prod.	matching typed constant/local operations; products flattened in field order	representation map agrees
integer primitive	same-width scalar operation	signed division only after explicit zero/overflow branch
floating primitive	matching strict scalar operation	target result related by NaN quotient; no relaxed/contraction operation
address/load	checked <i>i32</i> offset arithmetic followed by width-matched load	natural endpoint proof and bounds-before-address order
reserve/init/seal	no allocation instruction; assigned address, width-matched stores, then header stores	capability/prefix certificate and initialised-before-read
output store	width-matched stores in ABI field order	caller output interval and complete-success path
branch/cbr	structured branch depth $\lambda(L)$; scalar conditional	certificate maps every BIR successor exactly once
call/return	direct function index; status test and result continuation	compatible flattened ABI and decreasing source rank
fail	set injective nonzero status and branch to common epilogue	output ignored on nonzero status

References

- [1] Elvira Albert, Jesús Correas, Pablo Gordillo, Guillermo Román-Díez, and Albert Rubio. GASOL: Gas analysis and optimization for Ethereum smart contracts. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2020)*, volume 12079 of *Lecture Notes in Computer Science*, pages 118–125, 2020. doi: 10.1007/978-3-030-45237-7_7.
- [2] Elvira Albert, Jesús Correas, Pablo Gordillo, Guillermo Román-Díez, and Albert Rubio. Don’t run on fumes—parametric gas bounds for smart contracts. *Journal of Systems and Software*, 176:110923, 2021. doi: 10.1016/j.jss.2021.110923.
- [3] Massimo Bartoletti, Andrea Bracciali, Cristian Lepore, Alceste Scalas, and Roberto Zunino. A formal model of Algorand smart contracts. In *Financial Cryptography and Data Security*, volume 12674 of *Lecture Notes in Computer Science*, pages 93–114, 2021. doi: 10.1007/978-3-662-64322-8_5.
- [4] Zhuo Cai, Soroush Farokhnia, Amir Kafshdar Goharshady, and S. Hitarth. Asparagus: Automated synthesis of parametric gas upper-bounds for smart contracts. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA2):882–911, 2023. doi: 10.1145/3622829.
- [5] Quentin Carbonneaux, Jan Hoffmann, Tahina Ramananandro, and Zhong Shao. End-to-end verification of stack-space bounds for C programs. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2014. doi: 10.1145/2594291.2594301.
- [6] Quentin Carbonneaux, Jan Hoffmann, and Zhong Shao. Compositional certified resource bounds. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 467–478, 2015. doi: 10.1145/2737924.2737955.
- [7] Zhuo Zoey Chen, Johannes Åman Pohjola, and Christine Rizkallah. A verified cost model for Call-By-Push-Value. In *16th International Conference on Interactive Theorem Proving (ITP 2025)*, volume 352 of *Leibniz International Proceedings in Informatics*, pages 7:1–7:19, 2025. doi: 10.4230/LIPIcs.ITP.2025.7.
- [8] Ezgi Çiçek, Gilles Barthe, Marco Gaboardi, Deepak Garg, and Jan Hoffmann. Relational cost analysis. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, 2017. doi: 10.1145/3009837.3009858.
- [9] Ankush Das and Jan Hoffmann. ML for ML: Learning cost semantics by experiment. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2017)*, volume 10205 of *Lecture Notes in Computer Science*, pages 190–207, 2017. doi: 10.1007/978-3-662-54577-5_11.
- [10] Ankush Das and Shaz Qadeer. Exact and linear-time gas-cost analysis. In *Static Analysis (SAS 2020)*, volume 12389 of *Lecture Notes in Computer Science*, pages 333–356, 2020. doi: 10.1007/978-3-030-65474-0_15.
- [11] Ankush Das, Stephanie Balzer, Jan Hoffmann, Frank Pfenning, and Ishani Santurkar. Resource-aware session types for digital contracts. In *34th IEEE Computer Security Foundations Symposium (CSF)*, pages 1–16, 2021. doi: 10.1109/CSF51468.2021.00004.
- [12] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and JF Bastien. Bringing the Web up to speed with WebAssembly.

- In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 185–200, 2017. doi: 10.1145/3062341.3062363.
- [13] Jan Hoffmann and Martin Hofmann. Amortized resource analysis with polynomial potential. In *Programming Languages and Systems (ESOP 2010)*, volume 6012 of *Lecture Notes in Computer Science*, pages 287–306, 2010. doi: 10.1007/978-3-642-11957-6_16.
 - [14] Jan Hoffmann, Ankush Das, and Shu-Chun Weng. Towards automatic resource bound analysis for OCaml. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, pages 359–373, 2017. doi: 10.1145/3009837.3009842.
 - [15] Abhinav Jangda, Bobby Powers, Emery D. Berger, and Arjun Guha. Not so fast: Analyzing the performance of WebAssembly vs. native code. In *2019 USENIX Annual Technical Conference*, pages 107–120, 2019. URL <https://www.usenix.org/conference/atc19/presentation/jangda>.
 - [16] Sangeeta Kakati and Mats Brorsson. A cross-architecture evaluation of WebAssembly in the Cloud–Edge Continuum. In *2024 IEEE 24th International Symposium on Cluster, Cloud and Internet Computing*, 2024. doi: 10.1109/CCGrid59990.2024.00046.
 - [17] Byeongjee Kang, Harsh Desai, Limin Jia, and Brandon Lucia. WAMI: Compilation to WebAssembly through MLIR without losing abstraction. *arXiv preprint arXiv:2506.16048*, 2025. URL <https://arxiv.org/abs/2506.16048>.
 - [18] G. A. Kavvos, Edward Morehouse, Daniel R. Licata, and Norman Danner. Recurrence extraction for functional programs through Call-by-Push-Value. *Proceedings of the ACM on Programming Languages*, 4(POPL), 2020. doi: 10.1145/3371083.
 - [19] Markus Knecht and Burkhard Stiller. SCUR: Smart contracts with a static upper-bound on resource usage. In *2020 IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pages 1926–1934, 2020. doi: 10.1109/TrustCom50675.2020.00263.
 - [20] Tristan Knoth, Di Wang, Adam Reynolds, Jan Hoffmann, and Nadia Polikarpova. Liquid resource types. *Proceedings of the ACM on Programming Languages*, 4(ICFP), 2020. doi: 10.1145/3408988.
 - [21] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7): 107–115, 2009. doi: 10.1145/1538788.1538814.
 - [22] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. Alive2: Bounded translation validation for LLVM. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 2021. doi: 10.1145/3453483.3454030.
 - [23] Cynthia Marcelino, Noah Krennmair, Thomas Pusztai, and Stefan Nastic. Lumos: Performance characterization of WebAssembly as a serverless runtime in the Edge–Cloud Continuum. In *Proceedings of the 15th International Conference on the Internet of Things (IoT 2025)*, pages 113–121, 2025. doi: 10.1145/3770501.3770515.
 - [24] Konrad Moron and Stefan Wallentowitz. Benchmarking WebAssembly for embedded systems. *ACM Transactions on Architecture and Code Optimization*, 22(3):83:1–83:21, 2025. doi: 10.1145/3736169.

- [25] Russell O'Connor. Simplicity: A new language for blockchains. *arXiv preprint arXiv:1711.03028*, 2017. URL <https://arxiv.org/abs/1711.03028>.
- [26] Elena Panova, Valentin Volokitin, Anton Gorshkov, and Iosif Meyerov. Black–Scholes option pricing on Intel CPUs and GPUs: Implementation on SYCL and optimization techniques. *arXiv preprint arXiv:2204.13740*, 2022. URL <https://arxiv.org/abs/2204.13740>.
- [27] João Santos Reis, Paul Crocker, and Simão Melo de Sousa. Tezla, an intermediate representation for static analysis of Michelson smart contracts. *arXiv preprint arXiv:2005.11839*, 2020. URL <https://arxiv.org/abs/2005.11839>.
- [28] Ilya Sergey. The next 700 smart contract languages. In *Principles of Blockchain Systems*, pages 69–94. Springer, 2022. doi: 10.1007/978-3-031-01807-7_3.
- [29] Ilya Sergey, Vaivaswatha Nagaraj, Jacob Johannsen, Amrit Kumar, Anton Trunov, and Ken Chan Guan Hao. Safer smart contract programming with Scilla. *Proceedings of the ACM on Programming Languages*, 3(OOPSLA), 2019. doi: 10.1145/3360611.
- [30] Vilhelm Sjöberg, Kinnari Dave, Daniel Britten, Maria A. Schett, Xinyuan Sun, Qinshi Wang, Sean Noble Anderson, Steve Reeves, and Zhong Shao. Foundational verification of smart contracts through verified compilation. *arXiv preprint arXiv:2405.08348*, 2024. URL <https://arxiv.org/abs/2405.08348>.
- [31] Alexa VanHattum, Monica Pardeshi, Chris Fallin, Adrian Sampson, and Fraser Brown. Lightweight, modular verification for WebAssembly-to-native instruction selection. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2024. doi: 10.1145/3617232.3624862.
- [32] Stefan Wallentowitz, Bastian Kersting, and Dan Mihai Dumitriu. Potential of WebAssembly for embedded systems. In *2022 11th Mediterranean Conference on Embedded Computing*, 2022. doi: 10.1109/MECO55406.2022.9797106.
- [33] Peng Wang, Di Wang, and Adam Chlipala. TiML: A functional language for practical complexity analysis with invariants. *Proceedings of the ACM on Programming Languages*, 1(OOPSLA), 2017. doi: 10.1145/3133903.
- [34] Conrad Watt. Mechanising and verifying the WebAssembly specification. In *Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 53–65, 2018. doi: 10.1145/3167082.
- [35] Conrad Watt, Xiaojia Rao, Jean Pichon-Pharabod, Martin Bodin, and Philippa Gardner. Two mechanisations of WebAssembly 1.0. In *Formal Methods (FM 2021)*, volume 13047 of *Lecture Notes in Computer Science*, pages 61–79, 2021. doi: 10.1007/978-3-030-90870-6_4.